

مقياس UEF31

هندسة الحواسيب

Architecture des ordinateurs

- الفصل الأول: التنظيم العام للوحدة المركزية للحاسوب
- الفصل الثاني: البنية الداخلية للمعالج
- الفصل الثالث: دراسة حالة، المعالج Intel 8086
- الفصل الرابع: المعالجات ذات 32 و64 بت
- الفصل الخامس: بنية المعالجات الحديثة

الأستاذ: د. بن عبد الرزاق يوسف

المعامل: 3 **الأرصدة:** 5

التقييم: امتحان 60%، مراقبة مستمرة 40%

كيف تستعمل هذا الدرس؟

هذا الدرس مكتوب لك أنت، حتى لو كانت هذه أول مرة تفتح فيها «صندوق» الحاسوب. نبدأ من الأشياء البسيطة، ونصعد خطوة خطوة حتى نكتب برامج حقيقية بلغة التجميع (Assembly)، ثم نفهم كيف تعمل المعالجات الحديثة في هواتفنا وحواسيبنا.

الصناديق الملونة

ستجد في كل صفحة صناديق ملونة، لكل لون دور:

تعريف

كلمة جديدة مهمة، مع اسمها بالفرنسية والإنجليزية لأنك ستجدها في الكتب والمراجع.

تشبيه من الحياة اليومية

نشرح الفكرة الصعبة بشيء تعرفه من حياتك اليومية: المطبخ، المدرسة، الحافلة...

خطوة بخطوة

حساب أو تنفيذ نقوم به مرحلة بمرحلة، دون أن نقفز أي خطوة.

انتبه

خطأ يقع فيه كثير من الطلبة، أو تفصيل صغير يغيّر النتيجة.

هل تعلم؟

معلومة إضافية ممتعة، ليست ضرورية للامتحان لكنها تساعدك على الفهم.

البرامج في هذا الدرس

- كل برنامج مكتوب كاملًا، ومرفوق بجدول «**ماذا يفعل كل سطر؟**».
- برامج المعالج 8086 مكتوبة لبرنامج المحاكاة EMU8086، وبرامج 64 بت مكتوبة لـNASM على نظام Linux.
- كل البرامج **جُرّبت فعلًا**: ما تراه في الشاشة السوداء 🖵 هو ما يظهر عند التنفيذ، وجداول السجلات مأخوذة من التنفيذ الحقيقي.

انتبه

لا تكتفِ بالقراءة! انسخ كل برنامج في EMU8086، ونفّذه تعليمة بتعليمة (زر Single Step)، وراقب تغيّر السجلات. هكذا يُفهم هذا المقياس.

المحتويات

كيف تستعمل هذا الدرس؟ **1**

1 التنظيم العام للوحدة المركزية للحاسوب **8**

1.1 ما الحاسوب؟ 8

1.1.1 العتاد والبرمجيات 9

2.1.1 طبقات الحاسوب 9

3.1.1 كيف يعمل برنامج؟ 9

2.1 تاريخ الحواسيب باختصار 10

3.1 تذكير سريع: كيف يعدّ الحاسوب؟ 10

1.3.1 وحدات قياس الذاكرة 12

4.1 المكونات الأساسية للحاسوب 12

5.1 النموذجان الأساسيان: فون نيومان وهارفارد 13

1.5.1 نموذج فون نيومان (Von Neumann) 13

2.5.1 نموذج هارفارد (Harvard) 14

3.5.1 مقارنة واستعمالات 14

6.1 المعالج: الدور والمكونات 14

1.6.1 الساعة وسرعة المعالج 15

7.1 الذاكرة 15

1.7.1 كيف تبدو الذاكرة من الداخل؟ 15

2.7.1 عمليتان فقط: القراءة والكتابة 16

3.7.1 أنواع الذاكرة 16

4.7.1 حساب سعة الذاكرة 17

8.1 الحافلات 18

1.8.1 عرض الحافلات 19

9.1 تمارين 19

2 البنية الداخلية للمعالج **21**

1.2 نظرة إلى داخل المعالج 21

2.2 السجلات 22

1.2.2 لماذا السجلات مهمة للسرعة؟ 22

2.2.2 أنواع السجلات 23

3.2.2 سجل الأعلام 23
4.2.2 الفرق بين CF وOF 23
3.2 وحدة الحساب والمنطق (UAL) 24
1.3.2 كيف تجمع البوابات المنطقية؟ نصف المجمّع 24
2.3.2 المجمّع الكامل 25
3.3.2 مجمّع 4 بتات 25
4.3.2 والطرح؟ نستعمل المجمّع نفسه! 26
5.3.2 المقارنة بالطرح 26
6.3.2 تجربة حقيقية: كيف تتغير الأعلام؟ 26
4.2 وحدة التحكم (UC) 27
1.4.2 الأوامر الدقيقة نبضة بنبضة 28
2.4.2 نوعان من وحدات التحكم 28
5.2 مجموعة التعليمات 28
1.5.2 شكل التعليمة 28
2.5.2 كم تعليمة يمكن أن يملك المعالج؟ 29
3.5.2 عدد العناوين في التعليمة 29
4.5.2 أصناف التعليمات 30
5.5.2 CISC وRISC 30
6.2 أنماط العنونة 30
1.6.2 الأنماط الأساسية 30
2.6.2 متى نستعمل كل نمط؟ 31
3.6.2 الأنماط في برنامج حقيقي 31
7.2 مراحل تنفيذ تعليمة 32
1.7.2 مثال كامل خطوة بخطوة 33
8.2 تمارين 34

3 دراسة حالة: المعالج Intel 8086 36

1.3 عائلة 80x86 36
2.3 البنية الداخلية للمعالج 8086 37
3.3 سجلات المعالج 8086 37
1.3.3 السجلات العامة 37
2.3.3 سجلات المؤشرات والفهارس 38
3.3.3 سجلات المقاطع 38
4.3.3 مؤشر التعليمة وسجل الأعلام 39

4.3 العنونة المقطعية 39
1.4.3 المشكلة 39
2.4.3 الحل: المقطع والإزاحة 39
3.4.3 لماذا نضرب في 16؟ 40
4.4.3 الأزواج الافتراضية 41
5.3 أنماط العنونة في 8086 42
6.3 أول برنامج 43
1.6.3 الأدوات 43
2.6.3 البرنامج 43
3.6.3 التعريف بالمتغيرات 44
7.3 مراحل تنفيذ تعليمة داخل 8086 44
8.3 مجموعة التعليمات بالتفصيل 45
1.8.3 تعليمات النقل 45
2.8.3 التعليمات الحسابية 46
3.8.3 التعليمات المنطقية 48
4.8.3 التحكم في التدفق: القفزات 50
5.8.3 الحلقات والتكرار 53
6.8.3 توليد لغة الآلة 55
9.3 إدارة الإدخال والإخراج 57
1.9.3 المنافذ: `in` و`out` 57
2.9.3 خدمات DOS: المقاطعة `int 21h` 57
10.3 إدارة المكدس 58
1.10.3 لماذا نحتاج إلى المكدس؟ 59
11.3 الدوال والإجراءات 59
1.11.3 كيف يعرف `ret` إلى أين يعود؟ 59
2.11.3 إجراء يطبع عددًا بالعشري 60
3.11.3 تمرير المعاملات عبر المكدس 62
12.3 المقاطعات 63
1.12.3 أنواع المقاطعات 63
2.12.3 جدول أشعة المقاطعات 63
3.12.3 ماذا يفعل المعالج عند `int n`؟ 63
4.12.3 المقاطعة العتادية خطوة بخطوة 64
5.12.3 برنامج: مقاطعتنا الخاصة 64
13.3 تمارين 65

4 دراسة حالة: المعالجات ذات 32 و64 بتًا 67

1.4 من 16 إلى 32 ثم 64 بتًا 67

1.1.4 الفروق الأساسية 67

2.1.4 لماذا نحتاج إلى أكثر من 32 بتًا؟ 68

2.4 تطور المعمارية: من IA-32 إلى AMD64 68

1.2.4 أنماط عمل المعالج 68

3.4 السجلات في معالجات 32 و64 بتًا 68

1.3.4 السجلات المتداخلة 69

2.3.4 تجربة: ماذا يحدث للبتات الأخرى؟ 70

4.4 أنماط العنونة المتقدمة 71

1.4.4 من العنونة المقطعية إلى العنونة المسطحة 71

2.4.4 الشكل العام للعنوان 72

3.4.4 العنونة النسبية إلى RIP 74

5.4 مجموعة التعليمات 74

1.5.4 التعليمة نفسها على 16 و32 و64 بتًا 74

2.5.4 مقارنة: برنامج 32 بتًا وبرنامج 64 بتًا 75

3.5.4 ماذا يكتب المترجم؟ من C إلى لغة التجميع 76

4.5.4 SSE وAVX: حساب عدة أعداد مرة واحدة 77

6.4 دورة تنفيذ التعليمة وخط الأنابيب 79

1.6.4 المراحل الخمس 79

2.6.4 دون خط أنابيب ومع خط أنابيب 79

3.6.4 المخاطر: متى يتوقف خط الأنابيب؟ 80

4.6.4 مثال: `mov` و`add` و`jmp` في 64 بتًا 80

7.4 تمارين 81

5 بنية المعالجات الحديثة 83

1.5 المعالجات الحديثة: الاتجاهات والتطور 83

1.1.5 خمسون سنة من التقدم 83

2.1.5 جدار التردد 83

3.1.5 الاتجاهات الحالية 84

2.5 المعالجات متعددة الأنوية والتوازي 84

1.2.5 المفاهيم 84

2.2.5 Hyper-Threading: نواة واحدة لخيطين 85

3.2.5 تجربة: العمل نفسه على 1 و2 و4 و8 خيوط 85

4.2.5 قانون Amdahl: حدود التوازي 87
5.2.5 الفوائد والتحديات 88
3.5 الذاكرة المخبئية وتسلسل الذاكرة 88
1.3.5 المشكلة: الذاكرة بطيئة جدًا 88
2.3.5 تسلسل الذاكرة 89
3.3.5 تجربة: نرى طبقات الذاكرة بأعيننا 89
4.3.5 مبدأ المحلية: لماذا تنجح الذاكرة المخبئية؟ 91
5.3.5 تجربة: ترتيب القراءة يغيّر كل شيء 92
6.3.5 الإصابة والإخفاق وزمن الوصول المتوسط 93
7.3.5 أين نضع المعطاة في الذاكرة المخبئية؟ (التطابق) 93
8.3.5 الاستبدال والكتابة 94
4.5 تمارين 95

ا ملخص تعليمات 8086 **97**

ب EMU8086 وNASM: الفروق في الكتابة **99**

ج قاموس المصطلحات **100**

الفصل 1

التنظيم العام للوحدة المركزية للحاسوب

خلاصة: ماذا سنتعلم في هذا الفصل؟

- ما الحاسوب، وكيف تطوّر عبر التاريخ.
- المكونات الأساسية: المعالج، الذاكرة، وحدات الإدخال والإخراج.
- نموذجا Von Neumann وHarvard، والفرق بينهما.
- أنواع الذاكرة، وكيف نحسب سعتها.
- الحافلات الثلاث: حافلة المعطيات، حافلة العناوين، حافلة التحكم.

1.1 ما الحاسوب؟

تعريف

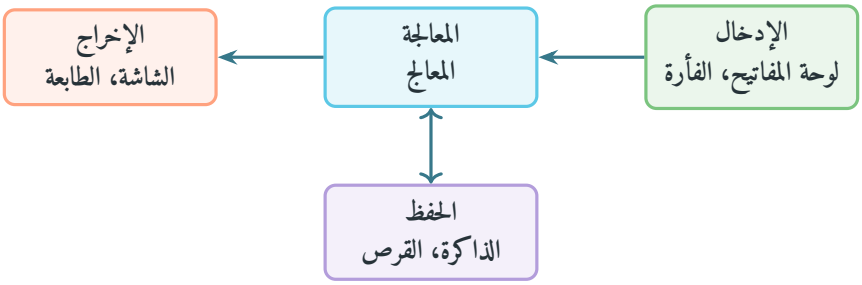
الحاسوب (Ordinateur / Computer): آلة إلكترونية تستقبل **معطيات**، وتعالجها حسب **برنامج** (قائمة من التعليمات)، ثم تعطي **نتائج**، ويمكنها أن تحفظ المعطيات والنتائج.

كل ما يفعله الحاسوب يمكن تلخيصه في أربعة أفعال: **يستقبل**، **يعالج**، **يحفظ**، **يُخرج**.

لنأخذ مثالًا بسيطًا: تستعمل تطبيق الآلة الحاسبة في حاسوبك لتحسب 25×4.

1) **الإدخال**: تكتب 25، ثم ×، ثم 4 بلوحة المفاتيح. كل ضغطة تتحول إلى رمز رقمي يصل إلى الحاسوب.
2) **الحفظ المؤقت**: يحفظ الحاسوب العددين في الذاكرة، لأنه يحتاج إليهما بعد قليل.
3) **المعالجة**: يقرأ المعالج العددين، وينفذ تعليمة الضرب، فيحصل على 100.
4) **الإخراج**: يرسل الحاسوب النتيجة إلى الشاشة فتراها.

كل برنامج تستعمله، مهما كان معقدًا (لعبة، متصفح، تطبيق صور)، هو في الحقيقة ملايين من هذه الخطوات الصغيرة، تُنفَّذ بسرعة كبيرة جدًا.



تشبيه من الحياة اليومية

الحاسوب يشبه **مطبخ مطعم**: المكونات (الخضر، اللحم) هي **المعطيات**، والوصفة هي **البرنامج**، والطباخ هو **المعالج**، والثلاجة هي **الذاكرة**، والطبق الجاهز هو **النتيجة**. الطباخ لا يقرر وحده: هو ينفذ الوصفة خطوة بخطوة.

1.1.1 العتاد والبرمجيات

- **العتاد** (Matériel / Hardware): كل ما يمكن لمسه: المعالج، الذاكرة، الشاشة، الأسلاك.
- **البرمجيات** (Logiciel / Software): البرامج التي تقول للعتاد ماذا يفعل: نظام التشغيل، المتصفح، الألعاب.

في هذا المقياس ندرس **العتاد من الداخل**، ونرى كيف ينفذ البرامج.

2.1.1 طبقات الحاسوب

يمكن أن نتخيل الحاسوب طبقات بعضها فوق بعض. كل طبقة تستعمل خدمات الطبقة التي تحتها، ولا تحتاج أن تعرف تفاصيلها:



في هذا الدرس نشتغل على **الطبقتين السفليين**: العتاد، ولغة التجميع التي تتكلم معه مباشرة.

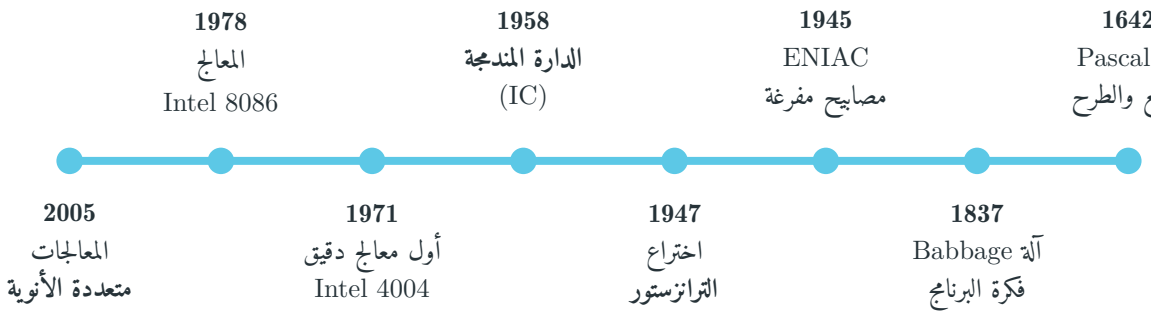
3.1.1 كيف يعمل برنامج؟

عندما تضغط مرتين على أيقونة برنامج، تحدث هذه المراحل:

1) البرنامج محفوظ على **القرص** (SSD أو HDD) في شكل ملف من الأصفار والآحاد.
2) نظام التشغيل **ينسخه في الذاكرة المركزية** (RAM)، لأن القرص بطيء جدًا والمعالج لا يقرأ منه مباشرة.
3) المعالج يقرأ التعليمات من الذاكرة **واحدة بعد الأخرى** وينفذها.
4) عندما تغلق البرنامج، تُحرَّر مساحته في الذاكرة لبرامج أخرى.

2.1 تاريخ الحواسيب باختصار

لم يظهر الحاسوب فجأة. هذه أهم المحطات:



الجيل	الفترة	التقنية الأساسية	مثال
الأول	1945–1955	المصابيح المفرغة	ENIAC (30 طنًا!)
الثاني	1955–1965	الترانزستور	IBM 7090
الثالث	1965–1980	الدارات المندمجة	IBM 360
الرابع	1980– الآن	المعالج الدقيق (VLSI)	الحاسوب الشخصي، الهاتف

كل جيل جديد جاء بتقنية تجعل المكونات **أصغر وأسرع وأقل استهلاكًا للطاقة**:

- **المصابيح المفرغة** كانت كبيرة، تسخن كثيرًا، وتحترق باستمرار؛ كان يجب تبديل بعضها كل يوم تقريبًا.
- **الترانزستور** يقوم بالعمل نفسه (مفتاح كهربائي يفتح ويغلق)، لكنه أصغر بمئات المرات ولا يحترق.
- **الدارة المندمجة** تضع آلاف الترانزستورات على قطعة صغيرة واحدة من السيليسيوم، بدل توصيلها بالأسلاك.
- **المعالج الدقيق** يضع المعالج **كله** في شريحة واحدة. من هنا ولد الحاسوب الشخصي ثم الهاتف الذكي.

تعريف

قانون مور (Loi de Moore / Moore's law): لاحظ Gordon Moore (أحد مؤسسي Intel) سنة 1965 أن عدد الترانزستورات في الشريحة **يتضاعف تقريبًا كل سنتين**. بقيت هذه الملاحظة صحيحة لأكثر من خمسين سنة: من 2300 ترانزستور في 4004 إلى عشرات المليارات اليوم.

هل تعلم؟

كان ENIAC يملأ قاعة كاملة ويستهلك 150 كيلوواط، بينما هاتفك أقوى منه بملايين المرات ويعمل ببطارية صغيرة. السبب: الترانزستورات أصبحت صغيرة جدًا، حتى صار المعالج الواحد يحتوي على مليارات منها.

3.1 تذكير سريع: كيف يعدّ الحاسوب؟

الحاسوب لا يعرف إلا حالتين كهربائيتين: **يمر التيار** أو **لا يمر**. نكتبهما 1 و0.

تعريف

- **البت** (Bit / Bit): أصغر وحدة معلومة، قيمته 0 أو 1.
- **البايت** (Octet / Byte): مجموعة من **8 بتات**.
- **الكلمة** (Mot / Word): عدد البتات التي يعالجها المعالج مرة واحدة (16 بتًا في المعالج 8086).

نكتب الأعداد في ثلاثة أنظمة: العشري (الأساس 10)، الثنائي (الأساس 2)، والست عشري (الأساس 16، أرقامه من 0 إلى 9 ثم A إلى F).

عشري	ثنائي	ست عشري	عشري	ثنائي	ست عشري
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

خطوة بخطوة: تحويل $2F_H$ إلى الثنائي وإلى العشري

1) كل رقم ست عشري يساوي **4 بتات**: $2 = 0010$ و$F = 1111$.
2) نضعهما جنبًا إلى جنب: $2F_H = 0010\,1111_2$.
3) إلى العشري: $2 \times 16 + 15 = 32 + 15 = 47$.

خطوة بخطوة: تحويل العدد الثنائي 1011 إلى العشري

كل بت له «وزن» هو قوة من قوى 2، تبدأ من اليمين بـ $2^0 = 1$:

البت	1	0	1	1
الوزن	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
البت × الوزن	8	0	2	1

نجمع: $8 + 0 + 2 + 1 = 11$. إذن $1011_2 = 11$.

خطوة بخطوة: تحويل العدد العشري 13 إلى الثنائي (القسمة المتتالية على 2)

1) $13 \div 2 = 6$، والباقي **1**.
2) $6 \div 2 = 3$، والباقي **0**.
3) $3 \div 2 = 1$، والباقي **1**.
4) $1 \div 2 = 0$، والباقي **1**. نتوقف لأن الحاصل أصبح 0.
5) نقرأ البواقي **من الأخير إلى الأول**: 1101_2. تحقق: $8 + 4 + 0 + 1 = 13$.

تشبيه من الحياة اليومية

لماذا نستعمل الست عشري؟ لأن العدد الثنائي طويل جدًا ومتعب للعين: $1010\,1111\,0011\,0110_2$. الست عشري يختصر كل 4 بتات في رقم واحد، فيصبح العدد $AF36_H$. هو «الاختزال» الذي يستعمله المبرمجون لكتابة الثنائي بسرعة، كما نكتب «د.» بدل «دكتور».

انتبه

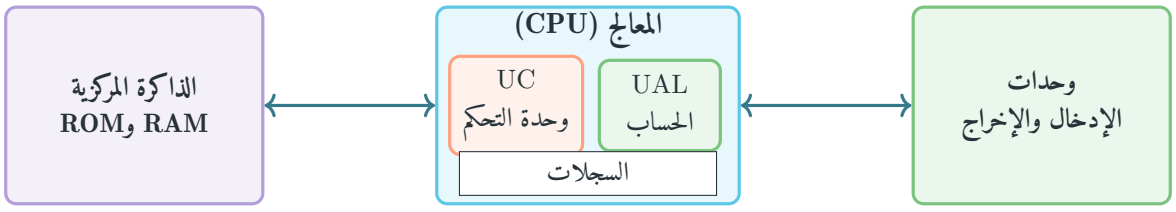
في لغة التجميع نكتب حرف `h` بعد العدد الست عشري (`2Fh`)، وحرف `b` بعد العدد الثنائي (`00101111b`). العدد دون حرف هو عدد عشري.

1.3.1 وحدات قياس الذاكرة

الوحدة	الرمز	القيمة
كيلوبايت	KB	$2^{10} = 1024$ بايت
ميغابايت	MB	$2^{20} = 1\,048\,576$ بايت
غيغابايت	GB	2^{30} بايت (حوالي مليار)
تيرابايت	TB	2^{40} بايت

4.1 المكونات الأساسية للحاسوب

داخل كل حاسوب، من أكبر خادم إلى أصغر ساعة ذكية، نجد ثلاثة أجزاء أساسية:



1) **المعالج** (Processeur / Processor, CPU): «دماغ» الحاسوب. يقرأ التعليمات وينفذها.
2) **الذاكرة المركزية** (Mémoire centrale / Main memory): تحفظ البرنامج الذي يعمل الآن ومعطياته.

3) **وحدات الإدخال والإخراج** (Périphériques d'E/S / I/O devices): تسمح للحاسوب بالتواصل مع العالم الخارجي.

خطوة بخطوة: رحلة حرف من لوحة المفاتيح إلى الشاشة

تضغط على المفتاح A في برنامج كتابة:

1) لوحة المفاتيح (**إدخال**) ترسل رمز المفتاح إلى الحاسوب.
2) المعالج يُقاطَع (يتوقف لحظة عن عمله)، فيقرأ الرمز، ويحفظه في **الذاكرة** داخل النص الذي تكتبه.
3) برنامج الكتابة يطلب من المعالج «رسم» الحرف: يحسب المعالج النقاط الضوئية التي تشكّل الحرف.
4) هذه النقاط تُرسل إلى بطاقة الشاشة، ثم إلى الشاشة (**إخراج**).

كل هذا يحدث في جزء صغير جدًا من الثانية، فتشعر أن الحرف ظهر فورًا.

تعريف

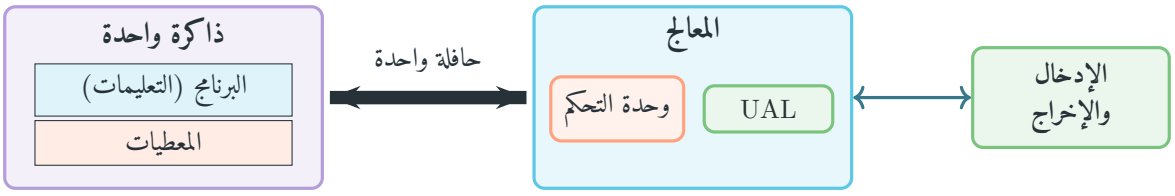
الوحدة المركزية (Unité centrale / Central unit): المعالج مع الذاكرة المركزية. هي الجزء الذي ينفذ البرامج فعلًا.

5.1 النموذجان الأساسيان: فون نيومان وهارفارد

1.5.1 نموذج فون نيومان (Von Neumann)

في سنة 1945 اقترح العالم John von Neumann فكرة بسيطة وعبقرية: نحفظ **البرنامج والمعطيات معًا في الذاكرة نفسها**. قبل ذلك، كان تغيير البرنامج يتطلب إعادة توصيل الأسلاك يدويًا!

حسب فون نيومان، يتكون الحاسوب من **خمس وحدات**: وحدة الإدخال، وحدة الإخراج، الذاكرة، وحدة الحساب والمنطق، ووحدة التحكم. والفكرة الأهم هي **البرنامج المخزَّن** (Programme enregistré / Stored program): التعليمات أعداد مثل المعطيات تمامًا، فنستطيع أن نحفظها في الذاكرة نفسها، ونقرأها ونغيّرها بالطريقة نفسها.



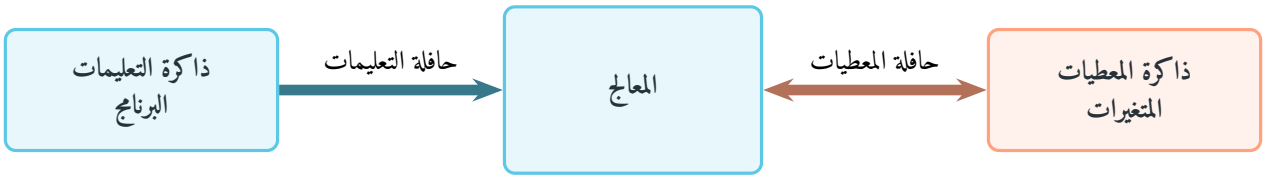
- **ميزته**: بسيط ومرن. نغيّر البرنامج بمجرد تحميل برنامج آخر في الذاكرة.
- **عيبه**: حافلة واحدة للتعليمات والمعطيات، فلا يمكن قراءة تعليمة ومعطاة في اللحظة نفسها. هذا الاختناق يسمى **عنق زجاجة فون نيومان** (Goulot de Von Neumann / Von Neumann bottleneck).

تشبيه من الحياة اليومية

تخيّل مكتبة لها **باب واحد ضيق**: الطلبة الذين يدخلون لإرجاع الكتب (المعطيات) والذين يدخلون لأخذ التعليمات (البرنامج) يمرون كلهم من الباب نفسه، واحدًا واحدًا. مهما كان الموظفون سريعين في الداخل، يبقى الباب هو ما يحدد السرعة. هذا هو عنق الزجاجة.

2.5.1 نموذج هارفارد (Harvard)

في هذا النموذج نستعمل **ذاكرتين منفصلتين**: واحدة للتعليمات وأخرى للمعطيات، ولكل واحدة حافلتها الخاصة.



- **ميزته**: أسرع، لأن المعالج يقرأ التعليمة التالية ويقرأ المعطاة في الوقت نفسه.
- **عيبه**: أعقد وأغلى، والمساحة المخصصة لكل ذاكرة ثابتة.

تشبيه من الحياة اليومية

في نموذج هارفارد للمكتبة **بابان**: باب لأخذ التعليمات، وباب للمعطيات. الطابوران يتقدمان في الوقت نفسه، فيمر الناس أسرع. لكن بناء بابين أغلى، وإذا امتلأ أحد الجانبين لا نستطيع استعمال مساحة الجانب الآخر.

3.5.1 مقارنة واستعمالات

	فون نيومان	هارفارد
الذاكرة	واحدة للبرنامج والمعطيات	ذاكرتان منفصلتان
الحافلات	حافلة واحدة مشتركة	حافلتان مستقلتان
السرعة	أبطأ (عنق الزجاجة)	أسرع (قراءتان معًا)
التعقيد	بسيط ورخيص	أعقد وأغلى
الاستعمال	الحواسيب الشخصية، الخوادم	المتحكمات الدقيقة (Arduino)، معالجات الإشارة (DSP)

هل تعلم؟

المعالجات الحديثة في حاسوبك تجمع بين النموذجين: الذاكرة الرئيسية (RAM) واحدة (فون نيومان)، لكن داخل المعالج توجد ذاكرتان صغيرتان سريعتان منفصلتان: واحدة للتعليمات وأخرى للمعطيات (هارفارد). نسمي ذلك **هارفارد المعدّل**، وسنعود إليه في الفصل الخامس.

6.1 المعالج: الدور والمكونات

تعريف

المعالج (Processeur, UCT / CPU): الدارة التي تنفذ تعليمات البرنامج واحدة تلو الأخرى.

يتكون المعالج من ثلاثة أجزاء رئيسية (سندرسها بالتفصيل في الفصل الثاني):

الجزء	دوره	التشبيه
وحدة التحكم (UC)	تقرأ التعليمة، تفهمها، وتعطي الأوامر لباقي الأجزاء	قائد الأوركسترا
وحدة الحساب والمنطق (UAL)	تنفذ العمليات الحسابية ($+$ $-$) والمنطقية (و، أو)	الآلة الحاسبة
السجلات (Registres)	ذاكرة صغيرة جدًا وسريعة جدًا داخل المعالج	ورقة المسودة على المكتب

1.6.1 الساعة وسرعة المعالج

يعمل المعالج على إيقاع **ساعة** (Horloge / Clock) ترسل نبضات منتظمة. في كل نبضة يتقدم المعالج خطوة صغيرة في عمله.

تعريف

التردد f: عدد النبضات في الثانية، ويقاس بالهرتز (Hz). **الدورة** T: مدة نبضة واحدة، و$T = \frac{1}{f}$.

خطوة بخطوة: معالج تردده 3 GHz

1) $f = 3\ \text{GHz} = 3 \times 10^9$ نبضة في الثانية.
2) $T = \frac{1}{3 \times 10^9} \approx 0.33 \times 10^{-9}$ ثانية، أي حوالي 0.33 نانوثانية.
3) في الوقت الذي ترمش فيه عينك (0.1 ثانية تقريبًا) يقوم هذا المعالج بـ 300 مليون دورة!

انتبه

التردد وحده لا يحدد سرعة المعالج. ما يهمنا هو **عدد التعليمات المنفذة في الثانية**. معالج بـ3 GHz يحتاج 4 دورات لكل تعليمة ينفذ $\frac{3\times10^9}{4} = 750$ مليون تعليمة في الثانية، بينما معالج بـ2 GHz ينفذ تعليمتين في كل دورة ينفذ 4×10^9 تعليمة، فهو أسرع رغم أن تردده أقل.

7.1 الذاكرة

1.7.1 كيف تبدو الذاكرة من الداخل؟

نتخيل الذاكرة مثل **عمارة كبيرة من الصناديق**. لكل صندوق رقم يسمى **العنوان**، وفي كل صندوق نحفظ بايتًا واحدًا.

العنوان	المحتوى
0000	3A
0001	FF
0002	07
0003	41
0004	00

العنوان 0003 يحتوي على 41_H.
العنوان لا يتغير أبدًا، أما المحتوى فيمكن تغييره.

تشبيه من الحياة اليومية

العنوان يشبه **رقم المنزل** في شارع، والمحتوى يشبه **الشخص الذي يسكن فيه**. رقم المنزل ثابت، أما السكان فيمكن أن يتغيروا.

2.7.1 عمليتان فقط: القراءة والكتابة

- **القراءة**: يرسل المعالج عنوانًا، فتعيد الذاكرة المحتوى (المحتوى لا يتغير).
- **الكتابة**: يرسل المعالج عنوانًا ومعطاة، فتحفظ الذاكرة المعطاة مكان القديمة.

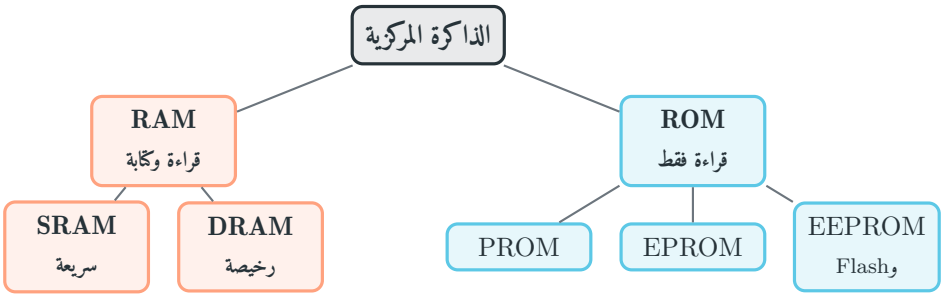
مثال

الخانة 0002 تحتوي على 07_H. ينفذ المعالج «اكتب 55_H في 0002». بعدها تحتوي الخانة على 55_H، و07_H ضاعت نهائيًا. لهذا يجب أن ننتبه أين نكتب: الكتابة في المكان الخطأ تمحو معطيات مهمة.

تعريف

ذاكرة الوصول العشوائي (Mémoire à accès aléatoire / Random Access Memory): «عشوائي» هنا تعني أننا نستطيع الوصول إلى **أي خانة مباشرة** وفي الزمن نفسه، دون المرور بالخانات التي قبلها، عكس شريط الكاسيت القديم الذي يجب لفّه حتى نصل إلى الأغنية.

3.7.1 أنواع الذاكرة



النوع	الوصف	عند انقطاع التيار	الاستعمال
SRAM	ترانزستورات فقط، سريعة جدًا وغالية	تُمحى	الذاكرة المخبئية داخل المعالج
DRAM	مكثّف صغير لكل بت، يجب «تنشيطه» باستمرار	تُمحى	الذاكرة الرئيسية (شرائط RAM)
ROM	تُكتب مرة واحدة في المصنع	تبقى	برامج الإقلاع القديمة
EPROM	تُمحى بالأشعة فوق البنفسجية ثم تُعاد كتابتها	تبقى	النماذج الأولية
EEPROM / Flash	تُمحى كهربائيًا	تبقى	BIOS/UEFI، المفاتيح USB، SSD

تشبيه من الحياة اليومية

SRAM مثل مفتاح الضوء: يبقى في وضعه (مشتعل أو مطفأ) ما دام التيار موجودًا. أما **DRAM** فمثل كأس فيه ثقب صغير: الماء (الشحنة الكهربائية) يتسرب ببطء، فيجب إعادة ملئه آلاف المرات في الثانية. هذه العملية تسمى **التنشيط** (refresh). لكن الكأس أصغر بكثير من المفتاح، لذلك نستطيع وضع مليارات منها في شريط ذاكرة رخيص.

هل تعلم؟

لماذا يحتاج الحاسوب إلى ROM أو Flash؟ عند تشغيله تكون RAM فارغة تمامًا. يجب أن يجد المعالج برنامجًا أول يبدأ به: برنامج الإقلاع (BIOS أو UEFI)، المحفوظ في ذاكرة لا تُمحى. هذا البرنامج يفحص العتاد، ثم يبحث عن نظام التشغيل على القرص ويحمّله في RAM.

تعريف

ذاكرة **متطايرة** (Volatile / Volatile) تفقد محتواها عند انقطاع التيار (RAM)، وذاكرة **غير متطايرة** (Non-volatile / Non-volatile) تحتفظ به (ROM، Flash).

4.7.1 حساب سعة الذاكرة

إذا كان للمعالج n خطًا (سلكًا) للعنوان، فيمكنه أن يكتب 2^n عنوانًا مختلفًا.

تعريف

عدد الخانات $= 2^n$ و حجم الخانة $\times$ عدد الخانات $= 2^n$ السعة

خطوة بخطوة: سعة ذاكرة المعالج 8086

للمعالج 8086 حافلة عناوين من **20** بتًا، وكل خانة تحتوي على بايت واحد.

1) عدد العناوين: 2^{20}.

2) $2^{20} = 2^{10} \times 2^{10} = 1024 \times 1024 = 1\,048\,576$ عنوانًا.

3) السعة $= 2^{20} \times 1$ بايت $= 1$ ميغابايت (1 MB).

خطوة بخطوة: كم خطًا للعنوان نحتاج لذاكرة 64 KB؟

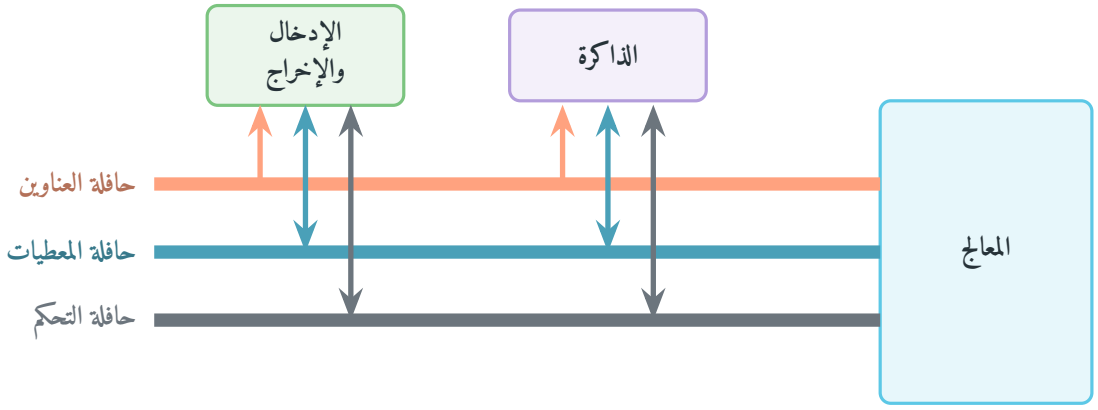
1) $64\ \text{KB} = 64 \times 1024 = 2^6 \times 2^{10} = 2^{16}$ بايت.

2) إذن $2^n = 2^{16}$، أي $n = 16$ خطًا.

8.1 الحافلات

تعريف

الحافلة (Bus / Bus): مجموعة من الأسلاك المتوازية، تنقل المعلومات بين المعالج والذاكرة ووحدات الإدخال والإخراج. كل سلك ينقل بتًا واحدًا في كل مرة.



الحافلة	الاتجاه	ماذا تنقل؟
العناوين (Address bus)	من المعالج فقط	رقم الخانة التي يريد المعالج قراءتها أو الكتابة فيها
المعطيات (Data bus)	في الاتجاهين	القيمة المقروءة أو المكتوبة
التحكم (Control bus)	في الاتجاهين	إشارات مثل: قراءة (RD)، كتابة (WR)، الساعة، المقاطعة

تشبيه من الحياة اليومية

تخيّل أنك تطلب طردًا من مكتب البريد: **العنوان** يخبر الموظف أي رف يقصد (حافلة العناوين)، **والطرد** نفسه هو المعطاة (حافلة المعطيات)، وكلمة «أعطني» أو «خذ» هي إشارة التحكم (قراءة أو كتابة).

خطوة بخطوة: المعالج يقرأ الخانة 0003

1) يضع المعالج العنوان 0003 على **حافلة العناوين**.
2) يرسل إشارة **قراءة** (RD) على **حافلة التحكم**.
3) تضع الذاكرة محتوى الخانة (41_H) على **حافلة المعطيات**.
4) يأخذ المعالج القيمة 41_H ويحفظها في أحد سجلاته.

1.8.1 عرض الحافلات

- **عرض حافلة العناوين** يحدد **سعة الذاكرة القصوى**: n خطًا تعطي 2^n عنوانًا.
- **عرض حافلة المعطيات** يحدد **كم بتًا ننقل في كل مرة**: حافلة من 16 بتًا تنقل بايتين في المرة الواحدة.

خطوة بخطوة: نقل 1000 بايت بحافلات مختلفة

1) حافلة معطيات من **8 بتات** تنقل بايتًا واحدًا في كل مرة: نحتاج إلى 1000 عملية نقل.
2) حافلة من **16** بتًا تنقل بايتين: $1000 \div 2 = 500$ عملية.
3) حافلة من **64** بتًا تنقل 8 بايتات: $1000 \div 8 = 125$ عملية فقط.

كلما كانت الحافلة أعرض، انتقلت المعطيات أسرع، كما يمر عدد أكبر من السيارات في طريق سيّار من 8 مسالك مقارنة بطريق من مسلك واحد.

المعالج	حافلة المعطيات	حافلة العناوين	الذاكرة القصوى
Intel 8086	16 بتًا	20 بتًا	$2^{20} = 1$ MB
Intel 80386	32 بتًا	32 بتًا	$2^{32} = 4$ GB
x86-64 الحديث	64 بتًا	48 بتًا (عمليًا)	$2^{48} = 256$ TB

9.1 تمارين

تمرين 1.1

حوّل الأعداد الآتية: (أ) $3C_H$ إلى العشري والثنائي. (ب) العدد العشري 200 إلى الست عشري.

الحل

(أ) $3C_H = 3 \times 16 + 12 = 60$، وبالثنائي $0011\ 1100_2$.
(ب) $200 = 12 \times 16 + 8$، والرقم 12 يُكتب C، إذن $200 = C8_H$.

تمرين 2.1

معالج له حافلة عناوين من 24 بتًا، وكل خانة ذاكرة تحتوي على بايت. ما السعة القصوى للذاكرة؟

الحل

$2^{24} = 2^4 \times 2^{20} = 16 \times 1\ \text{MB} = 16\ \text{MB}$.

تمرين 3.1

نريد ذاكرة سعتها 4 GB. كم خطًا تحتاج حافلة العناوين؟

الحل

$4\ \text{GB} = 2^2 \times 2^{30} = 2^{32}$ بايت، إذن نحتاج إلى **32 خطًا.**

تمرين 4.1

أجب بصحيح أو خطأ مع التعليل: (أ) في نموذج فون نيومان، يُحفظ البرنامج في ذاكرة مختلفة عن المعطيات. (ب) حافلة العناوين تنقل المعلومات في الاتجاهين. (ج) ذاكرة DRAM تحتفظ بمحتواها بعد إطفاء الحاسوب.

الحل

(أ) **خطأ**: في نموذج فون نيومان ذاكرة واحدة للبرنامج والمعطيات؛ الفصل خاصية نموذج هارفارد.
(ب) **خطأ**: المعالج وحده يرسل العناوين، فهي في اتجاه واحد.
(ج) **خطأ**: DRAM ذاكرة متطايرة، تُمحى عند انقطاع التيار.

تمرين 5.1

معالج تردده 2 GHz، وتحتاج كل تعليمة في المتوسط إلى 4 دورات. كم تعليمة ينفذ في الثانية؟

الحل

عدد الدورات في الثانية $= 2 \times 10^9$. عدد التعليمات $= \dfrac{2 \times 10^9}{4} = 0.5 \times 10^9$، أي **500 مليون تعليمة في الثانية.**

الفصل 2

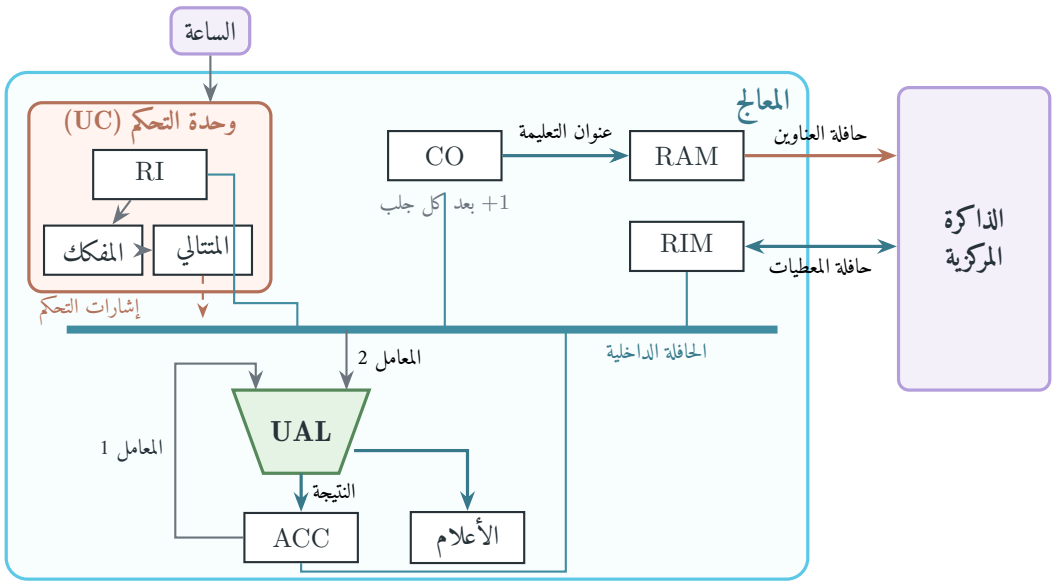
البنية الداخلية للمعالج

خلاصة: ماذا سنتعلم في هذا الفصل؟

- ما الموجود داخل المعالج، ودور كل جزء.
- السجلات وأنواعها.
- وحدة الحساب والمنطق: كيف يجمع المعالج عددين بالبوابات المنطقية، وما الأعلام.
- وحدة التحكم: من يقرر ماذا يحدث في كل لحظة.
- مجموعة التعليمات وأنماط العنونة.
- المراحل التي تمر بها كل تعليمة: الجلب، فك الترميز، التنفيذ، الكتابة.

1.2 نظرة إلى داخل المعالج

لنفتح المعالج ونرى ما بداخله. الشكل الآتي يمثل معالجًا بسيطًا، لكنه يحتوي على كل الأفكار الأساسية الموجودة في المعالجات الحقيقية.



العنصر	دوره في جملة واحدة
CO (العدّاد البرنامجي)	يحفظ عنوان **التعليمة القادمة**
RI (سجل التعليمة)	يحفظ **التعليمة الحالية** التي نفذها الآن
RAM (سجل عنوان الذاكرة)	يحفظ العنوان الذي نرسله إلى الذاكرة
RIM (سجل معطيات الذاكرة)	يحفظ المعطاة القادمة من الذاكرة أو الذاهبة إليها
ACC (المرِكّم)	يحفظ نتيجة الحساب
الأعلام	تحفظ معلومات عن آخر نتيجة: هل هي صفر؟ سالبة؟ ...
UAL	تنفذ العمليات الحسابية والمنطقية
المفكك والمتتالي	يفهمان التعليمة ويرسلان الأوامر بالترتيب الصحيح

انتبه

لا تخلط بين RAM الذاكرة (Random Access Memory) وRAM السجل (Registre d'Adresse Mémoire) في الكتب الفرنسية. في الإنجليزية يسمى هذا السجل MAR، ويسمى RIM سجل MDR، ويسمى CO سجل PC (Program Counter)، وفي المعالج 8086 نسميه IP.

2.2 السجلات

تعريف

السجل (Registre / Register): خانة ذاكرة صغيرة جدًا (8 أو 16 أو 32 أو 64 بتًا) موجودة **داخل المعالج**. هي أسرع ذاكرة في الحاسوب كله.

تشبيه من الحياة اليومية

عندما تحل تمرينًا في الرياضيات، لا تذهب إلى المكتبة في كل مرة لتقرأ عددًا! تكتب الأعداد التي تحتاجها على **ورقة المسودة** أمامك. السجلات هي ورقة المسودة للمعالج، والذاكرة هي المكتبة.

1.2.2 لماذا السجلات مهمة للسرعة؟

المكان	زمن الوصول التقريبي	لو كانت الدورة ثانية واحدة...
سجل	أقل من 1 ns	ثانية واحدة
الذاكرة الرئيسية RAM	حوالي 80 ns	دقيقة ونصف تقريبًا
قرص SSD	حوالي 100 µs	أكثر من يوم

لهذا نضع في السجلات المعطيات التي نستعملها كثيرًا، ونقلل من الذهاب إلى الذاكرة.

2.2.2 أنواع السجلات

1) **سجلات المعطيات** (Registres de données / Data registers): تحفظ الأعداد التي نحسب بها (مثل AX، BX في المعالج 8086).
2) **سجلات العناوين** (Registres d'adresse / Address registers): تحفظ عناوين في الذاكرة (مثل SI، DI، SP).
3) **سجلات التحكم والحالة** (Registres de contrôle / Control registers): تساعد المعالج على تنظيم عمله: CO، RI، وسجل الأعلام.

مثال: السجلات أثناء العمل

المعالج ينفذ الآن التعليمة الموجودة في العنوان 11، وهي «اجمع ACC مع محتوى الخانة 21».

- CO يحتوي على 12: عنوان التعليمة **القادمة**، لا الحالية.
- RI يحتوي على التعليمة الحالية نفسها، لكي تقرأها وحدة التحكم طيلة تنفيذها.
- RAM يحتوي على 21: العنوان الذي نرسله الآن إلى الذاكرة.
- RIM يستقبل محتوى الخانة 21 القادم من الذاكرة.
- ACC يحتوي على العدد الأول، وسيستقبل النتيجة.

كل سجل له دور واضح، ولهذا تسمى سجلات خاصة، عكس السجلات العامة التي يستعملها المبرمج كما يشاء.

3.2.2 سجل الأعلام

تعريف

العلم (Indicateur, drapeau / Flag): بت واحد تضعه وحدة الحساب في 1 أو 0 حسب نتيجة آخر عملية. نستعمل الأعلام بعد ذلك لاتخاذ القرارات (مثل: «إذا كانت النتيجة صفرًا، اقفز»).

العلم	يصبح 1 عندما...	مثال على 8 بتات
ZF (الصفر)	تكون النتيجة **صفرًا**	$5 - 5 = 0$
CF (الحمل)	يخرج حمل من آخر بت (النتيجة أكبر من السعة)	$200 + 100 = 300 > 255$
SF (الإشارة)	يكون البت الأخير للنتيجة 1 (عدد سالب)	$3 - 7 = -4$
OF (الفيضان)	تكون النتيجة خاطئة إذا اعتبرنا الأعداد بإشارة	$100 + 50 = 150 > 127$

4.2.2 الفرق بين CF وOF

البايت نفسه يمكن قراءته بطريقتين، والمعالج لا يعرف أي طريقة يقصدها المبرمج:

- **دون إشارة** (unsigned): القيم من 0 إلى 255.
- **بإشارة** (signed): القيم من -128 إلى $+127$. البت الأخير (البت 7) يدل على الإشارة: 1 = سالب.

لذلك يحسب المعالج العلمين معًا: CF يهم من يعمل بأعداد دون إشارة، وOF يهم من يعمل بأعداد بإشارة.

خطوة بخطوة: $50 + 100$ على 8 بتات

1) $100 = 0110\,0100_2$ و$50 = 0011\,0010_2$.
2) المجموع: $150 = 1001\,0110_2$ (دون إشارة). لا حمل من البت 7، إذن $CF = 0$: صحيح لمن يعمل دون إشارة.
3) لكن البت 7 أصبح 1! من يقرأ بإشارة يرى عددًا سالبًا: $150 - 256 = -106$.
4) جمعنا عددين موجبين فحصلنا على عدد سالب: هذا مستحيل، إذن النتيجة خاطئة بإشارة، و$OF = 1$.

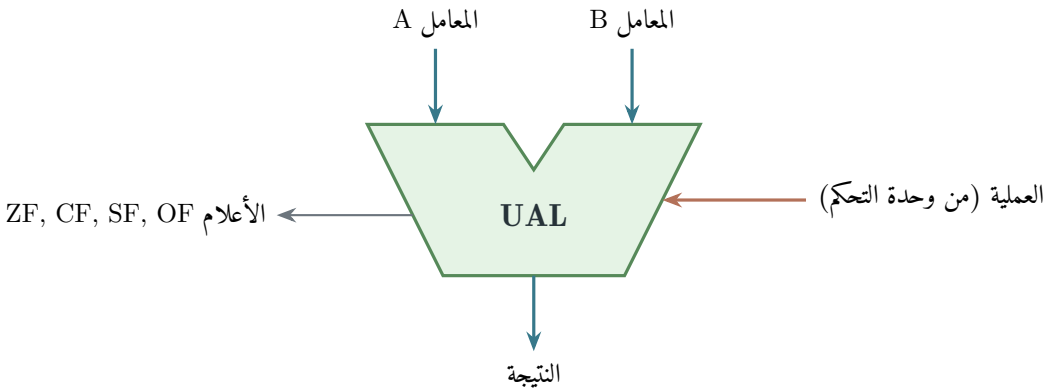
انتبه

قاعدة سهلة لـOF: يحدث الفيضان عندما نجمع عددين **من الإشارة نفسها** فنحصل على نتيجة **من الإشارة الأخرى**. جمع موجب مع سالب لا يسبب فيضانًا أبدًا.

3.2 وحدة الحساب والمنطق (UAL)

تعريف

وحدة الحساب والمنطق (Unité Arithmétique et Logique, UAL / Arithmetic Logic Unit, ALU): الجزء الذي ينفذ العمليات الحسابية (الجمع، الطرح، المقارنة...) والمنطقية (AND، OR، XOR، NOT، الإزاحة).



1.3.2 كيف تجمع البوابات المنطقية؟ نصف المجمّع

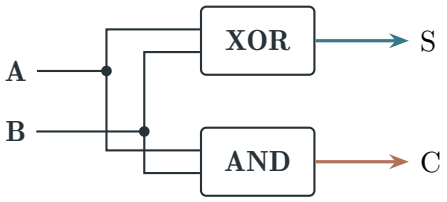
لنجمع بتين A وB. هناك أربع حالات فقط:

A	B	المجموع S	الحمل C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

نلاحظ أن: S يساوي 1 عندما يكون بت واحد فقط يساوي 1، وهذه هي البوابة XOR. وC يساوي 1 عندما يكون البتان يساويان 1، وهذه هي البوابة AND.

تعريف

نصف المجمّع (Demi-additionneur / Half adder): $S = A \oplus B$ و $C = A \cdot B$



2.3.2 المجمّع الكامل

لجمع أعداد من عدة بتات، نحتاج أن نضيف **الحمل القادم** من البت السابق. المجمّع الكامل له ثلاثة مداخل: A وB والحمل الداخل C_{in}.

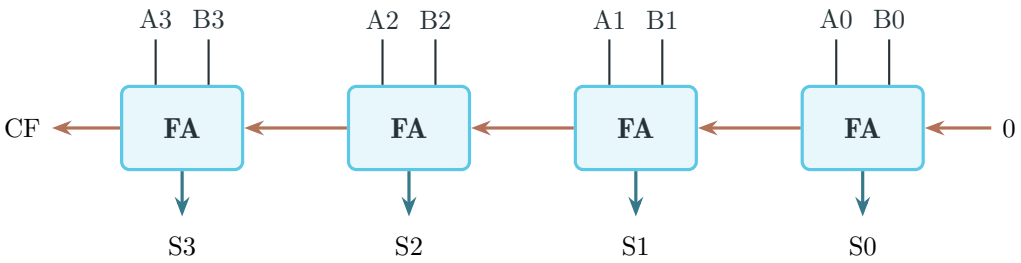
تعريف

المجمّع الكامل (Additionneur complet / Full adder): $S = A \oplus B \oplus C_{in}$ و $C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$

C_{out}	S	C_{in}	B	A
0	0	0	0	0
0	1	1	0	0
0	1	0	1	0
1	0	1	1	0
0	1	0	0	1
1	0	1	0	1
1	0	0	1	1
1	1	1	1	1

3.3.2 مجمّع 4 بتات

نضع 4 مجمّعات كاملة جنبًا إلى جنب: الحمل الخارج من كل واحد يدخل في الذي بعده، تمامًا كما نفعل عند الجمع بالورقة والقلم.



خطوة بخطوة: جمع $0110 + 0111$ (أي $6 + 7$) بالمجمّع

البت	A	B	C_{in}	S	C_{out}
0	1	0	0	1	0
1	1	1	0	0	1
2	1	1	1	1	1
3	0	0	1	1	0

نقرأ S من البت 3 إلى البت 0: $1101 = 13$، وهو فعلًا $6 + 7$. الحمل الأخير 0، إذن $\text{CF} = 0$.

4.3.2 والطرح؟ نستعمل المجمّع نفسه!

لا يحتاج المعالج إلى دارة خاصة للطرح. يستعمل فكرة **المتمم لاثنين** (Complément à deux / Two's complement): $A - B = A + (-B)$، و$-B$ نحصل عليه بقلب كل بتات B ثم إضافة 1.

خطوة بخطوة: $5 - 7$ على 4 بتات

1) $5 = 0101$. نقلب البتات: 1010. نضيف 1: 1011. إذن $-5 = 1011$.
2) نجمع: $0111 + 1011 = 1\,0010$. الحمل الخارج نتركه، فيبقى $0010 = 2$.
3) النتيجة $5 - 7 = 2$. صحيح! والدارة نفسها جمعت وطرحت.

5.3.2 المقارنة بالطرح

لا يحتاج المعالج إلى دارة خاصة للمقارنة: ليقارن A وB يحسب $A - B$ **دون أن يحفظ النتيجة**، ثم ينظر إلى الأعلام:

- إذا كان $\text{ZF} = 1$ فإن $A = B$.
- إذا كان $\text{CF} = 1$ (حدث «استلاف») فإن $A < B$ (للأعداد دون إشارة).

6.3.2 تجربة حقيقية: كيف تتغير الأعلام؟

نفذنا البرنامج الآتي على المعالج 8086. لا تقلق إن لم تفهم كل التعليمات الآن، سندرسها في الفصل الثالث؛ ركّز فقط على الأعلام.

```
1  org 100h
2
3      mov al, 200       ; AL = 200
4      add al, 100       ; 300 does not fit in 8 bits -> carry
5      mov al, 100       ; AL = 100
6      add al, 50        ; 150: too big for a signed byte -> overflow
7      mov al, 5
8      sub al, 5         ; result 0 -> zero flag
9      mov al, 3
10     sub al, 7         ; negative result -> sign flag and borrow
11     ret
```

OF	SF	ZF	CF	AL	التعليمة	السطر
0	0	0	1	$2C_H = 44$	$(200 + 100)$ `add al, 100`	4
0	0	0	1	$64_H = 100$	`mov al, 100`	5
1	1	0	0	$96_H = 150$	$(100 + 50)$ `add al, 50`	6
0	0	1	0	$00_H = 0$	$(5 - 5)$ `sub al, 5`	8
0	1	0	1	$FC_H = 252$	$(3 - 7)$ `sub al, 7`	10

خطوة بخطوة: لماذا $100 + 200$ أعطى 44؟

1) $200 + 100 = 300$، لكن AL يحتوي على 8 بتات فقط، أي من 0 إلى 255.

2) $300 - 256 = 44$: يبقى في AL العدد 44، و«الـ256» التي لم تجد مكانًا تذهب إلى العلم CF، فيصبح $CF = 1$.

انتبه

لاحظ السطر 5: التعليمة `mov` **لا تغيّر الأعلام**، لذلك بقي $CF = 1$. فقط تعليمات الحساب والمنطق تغيّر الأعلام.

4.2 وحدة التحكم (UC)

تعريف

وحدة التحكم (Unité de Commande, UC / Control Unit): الجزء الذي يقرأ التعليمات ويفهمها، ثم يرسل في كل نبضة ساعة **إشارات** إلى باقي الأجزاء (افتح هذا السجل، اجمع، اقرأ من الذاكرة...) بالترتيب الصحيح.

تشبيه من الحياة اليومية

وحدة التحكم هي **قائد الأوركسترا**: لا يعزف أي آلة بنفسه، لكنه يقول لكل عازف متى يبدأ ومتى يتوقف. بدونه يعزف الجميع في الوقت نفسه فتصبح ضوضاء.

مكوناتها الأساسية:

- **سجل التعليمة** (RI): يحفظ التعليمة الحالية.
- **المفكك** (Décodeur / Decoder): يقرأ الرمز العملي (opcode) ويعرف ما المطلوب: جمع؟ نقل؟ قفز؟
- **المتتالي** (Séquenceur / Sequencer): يرسل الإشارات المناسبة، واحدة بعد الأخرى، على إيقاع الساعة.

1.4.2 الأوامر الدقيقة نبضة بنبضة

كل مرحلة من مراحل التعليمة تنقسم إلى خطوات صغيرة جدًا تسمى **العمليات الدقيقة** (Micro-opérations / Micro-operations). في كل نبضة ساعة يرسل المتتالي إشارات تفتح «أبوابًا» بين السجلات. هذا مثال لجلب تعليمة:

النبضة	العملية الدقيقة	الإشارة التي يرسلها المتتالي
T1	RAM ← CO	«انسخ CO في RAM»
T2	RIM ← [RAM]، CO ← CO + 1	«اقرأ من الذاكرة» و«زِد CO»
T3	RI ← RIM	«انسخ RIM في RI»
T4...	حسب التعليمة	المفكك يختار الإشارات التالية

نلاحظ أن الجلب متشابه في كل التعليمات، أما ما بعده فيختلف حسب الرمز العملي.

2.4.2 نوعان من وحدات التحكم

	المبرمجة بالأسلاك (câblée)	المبرمجة دقيقًا (microprogrammée)
المبدأ	دارة منطقية ثابتة تولّد الإشارات	كل تعليمة = برنامج صغير مخزن في ذاكرة داخلية
السرعة	سريعة جدًا	أبطأ قليلًا
التعديل	صعب (يجب تغيير الدارة)	سهل (نغيّر البرنامج الدقيق)
أين نجدها	معالجات RISC	معالجات CISC مثل 8086

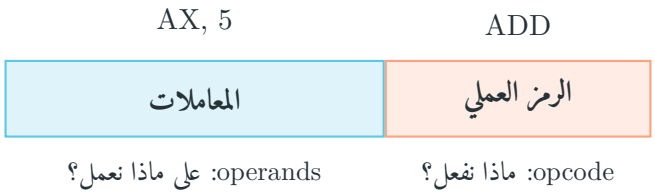
5.2 مجموعة التعليمات

تعريف

مجموعة التعليمات (Jeu d'instructions / Instruction Set): قائمة كل التعليمات التي يعرف المعالج تنفيذها. كل معالج له مجموعته الخاصة.

1.5.2 شكل التعليمة

كل تعليمة في لغة الآلة تتكون من جزأين:



تعريف

لغة التجميع (Langage d'assemblage / Assembly language): كتابة التعليمات بكلمات قصيرة يفهمها الإنسان (`ADD`، `MOV`) بدل الأعداد الثنائية. برنامج **المجمِّع** (Assembleur / Assembler) يحوّلها إلى لغة الآلة (أصفار وآحاد).

مثال

في المعالج 8086، التعليمة `add al, 5` تُكتب في لغة الآلة بالبايتين 05 04، أي 0101 0000 0100 0000: البايت الأول هو الرمز العملي «أضف قيمة فورية إلى AL»، والثاني هو العدد 5. المبرمج يكتب `add al, 5`، والمجمِّع يكتب الأصفار والآحاد بدلًا منه (سنرى ذلك بالتفصيل في الفصل الثالث).

2.5.2 كم تعليمة يمكن أن يملك المعالج؟

إذا كان الرمز العملي مكتوبًا على k بتات، فهناك 2^k رمزًا مختلفًا، أي 2^k تعليمة على الأكثر.

خطوة بخطوة

رمز عملي من 6 بتات: $2^6 = 64$ تعليمة ممكنة. رمز من 8 بتات: $2^8 = 256$ تعليمة.

3.5.2 عدد العناوين في التعليمة

لنحسب $A = B + C$ في أربعة أنواع من الآلات. كلما قلّ عدد المعاملات في التعليمة، قصرت التعليمة، لكن احتجنا إلى تعليمات أكثر:

نوع الآلة	البرنامج	عدد التعليمات
3 عناوين	`ADD A, B, C`	1
عنوانان	`MOV A, B` ثم `ADD A, C`	2
عنوان واحد (بمركّم)	`LOAD B` ثم `ADD C` ثم `STORE A`	3
دون عنوان (بمكدس)	`PUSH B`، `PUSH C`، `ADD`، `POP A`	4

المعالج 8086 آلة **بعنوانين**: `add ax, bx` تعني $AX \leftarrow AX + BX$.

4.5.2 أصناف التعليمات

الصنف	الدور	أمثلة (8086)
نقل المعطيات	نسخ قيمة من مكان إلى آخر	MOV، PUSH، POP
الحساب	الجمع، الطرح، الضرب، القسمة	ADD، SUB، MUL، INC
المنطق	العمليات على البتات	AND، OR، XOR، SHL
التحكم في التدفق	القفز وتغيير ترتيب التنفيذ	JMP، JE، CALL، RET
الإدخال والإخراج	التواصل مع الأجهزة	IN، OUT، INT

5.5.2 CISC و RISC

- **CISC** (Complex Instruction Set Computer): تعليمات كثيرة ومعقدة، بعضها يقوم بعمل كبير. مثال: Intel x86.
- **RISC** (Reduced Instruction Set Computer): تعليمات قليلة وبسيطة وسريعة، كلها بالطول نفسه. مثال: ARM في الهواتف، RISC-V.

6.2 أنماط العنونة

تعريف

نمط العنونة (Mode d'adressage / Addressing mode): الطريقة التي نحدد بها **أين توجد المعطاة** التي تعمل عليها التعليمة.

تشبيه من الحياة اليومية

تريد أن تعطي صديقك كتابًا: **(1)** تعطيه الكتاب في يده (فوري)، **(2)** تقول له: «الكتاب على الرف رقم 12» (مباشر)، **(3)** تقول له: «انظر في الورقة الموجودة في جيبي، فيها رقم الرف» (غير مباشر)، **(4)** تقول: «الرف الثالث بعد الرف رقم 10» (مفهرس).

1.6.2 الأنماط الأساسية

النمط	مثال 8086	أين المعطاة؟
فوري (immédiat)	`mov ax, 5`	داخل التعليمة نفسها
سجلي (registre)	`mov bx, ax`	في سجل
مباشر (direct)	`mov cx, [val]`	في الذاكرة، وعنوانها مكتوب في التعليمة
غير مباشر (indirect)	`mov dx, [si]`	في الذاكرة، وعنوانها موجود في سجل
مفهرس (indexé)	`mov ax, [tab+di]`	العنوان = عنوان بداية + قيمة سجل الفهرس
نسبي (relatif)	`jmp suite`	العنوان = العنوان الحالي + إزاحة (للقفز)

2.6.2 متى نستعمل كل نمط؟

- **الفوري**: للثوابت (عدد الدورات، القيمة الأولية). أسرع نمط، لأن القيمة وصلت مع التعليمة، فلا نذهب إلى الذاكرة. لكن لا يمكن تغييرها أثناء التنفيذ.
- **السجلي**: للحسابات السريعة بين قيم موجودة داخل المعالج. سريع جدًا أيضًا.
- **المباشر**: للمتغيرات التي لها مكان ثابت في الذاكرة (مثل متغير عام). نحتاج إلى قراءة واحدة من الذاكرة.
- **غير المباشر**: عندما لا نعرف العنوان عند كتابة البرنامج، بل نحسبه أثناء التنفيذ. هو أساس **المؤشرات** في لغة C.
- **المفهرس**: للجداول: نثبت عنوان البداية، ونغيّر الفهرس في كل دورة من الحلقة لنمر على كل العناصر.
- **النسبي**: للقفزات. بما أن المسافة نسبية، يعمل البرنامج حتى لو حُمّل في مكان آخر من الذاكرة.

خطوة بخطوة: كم مرة نذهب إلى الذاكرة؟

التعليمة	قراءات الذاكرة لجلب المعطاة
(فوري) `mov ax, 5`	0
(سجلي) `mov ax, bx`	0
(مباشر) `mov ax, [val]`	1
(غير مباشر بسجل) `mov ax, [si]`	1
غير مباشر بالذاكرة (العنوان نفسه في الذاكرة)	2

كل ذهاب إلى الذاكرة يكلف وقتًا، لذلك يحاول المبرمج إبقاء المعطيات المهمة في السجلات.

3.6.2 الأنماط في برنامج حقيقي

```
1  org 100h              ; .COM program starts at offset 100h
2
3      mov ax, 5         ; immediate : the value 5 is inside the instruction
4      mov bx, ax        ; register  : copy register AX into BX
5      mov cx, [val]     ; direct    : read memory at the address of val
6      lea si, val       ; SI = address of val
7      mov dx, [si]      ; indirect  : read memory at the address in SI
8      mov di, 4         ; DI = 4 (two words further)
9      mov ax, [tab+di]  ; indexed   : read tab + 4 = third element
10     ret               ; end of program
11
12 val dw 1234h          ; a word in memory
13 tab dw 10, 20, 30     ; an array of 3 words
```

	الذاكرة	
val	1234	0117
tab	000A	0119
	0014	011B
	001E	011D

+4

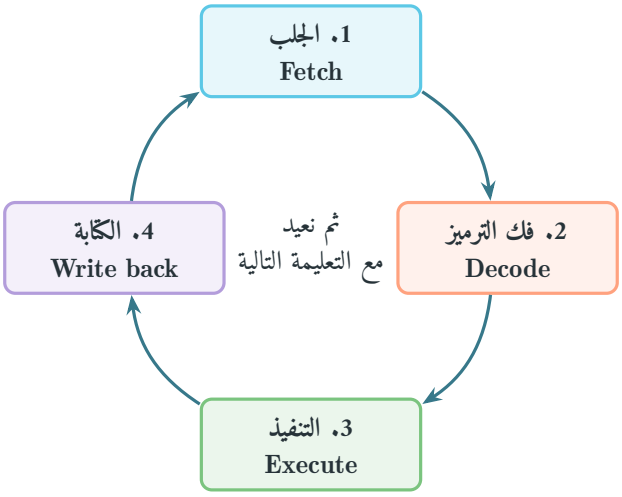
السطر	ماذا يفعل؟
1	برنامج من النوع COM.: يبدأ في العنوان 100_H.
3	**فوري**: العدد 5 مكتوب داخل التعليمة. النتيجة: AX = 0005.
4	**سجلي**: نسخ AX في BX. النتيجة: BX = 0005.
5	**مباشر**: نقرأ الذاكرة في عنوان `val` (0117_H). النتيجة: CX = 1234.
6	`lea` تضع **عنوان** `val` (لا قيمته) في SI: SI = 0117.
7	**غير مباشر**: نقرأ الذاكرة في العنوان الموجود في SI. النتيجة: DX = 1234.
8	DI = 4: كل عنصر من `tab` كلمة (بايتان)، إذن العنصر الثالث على بعد 4 بايتات.
9	**مفهرس**: العنوان $0119_H + 4 = 011D_H$، وفيه 30. النتيجة: AX = 001E.
10	نهاية البرنامج والعودة إلى النظام.

انتبه

القوسان `[ ]` يعنيان دائمًا: «**محتوى الذاكرة** في هذا العنوان». `mov dx, si` ينسخ العدد 0117_H نفسه، أما `mov dx, [si]` فيقرأ ما هو مخزن في العنوان 0117_H.

7.2 مراحل تنفيذ تعليمة

كل تعليمة، مهما كانت بسيطة، تمر بالمراحل نفسها. نسمي ذلك **دورة التعليمة** (Cycle d'instruction / Instruction cycle).



1) **الجلب**: يُرسل CO إلى RAM، نقرأ الذاكرة، تصل التعليمة إلى RIM ثم إلى RI. ثم يزيد CO ليشير إلى التعليمة التالية.
2) **فك الترميز**: يقرأ المفكك الرمز العملي الموجود في RI ويحدد المعاملات.
3) **التنفيذ**: إن احتجنا معطاة من الذاكرة نقرؤها، ثم تحسب UAL النتيجة.

4) **الكتابة**: نحفظ النتيجة في سجل أو في الذاكرة، ونحدّث الأعلام.

1.7.2 مثال كامل خطوة بخطوة

نأخذ آلة بسيطة بمركّم ACC، ونريد حساب $z = x + y$. المتغير x في العنوان 20، وy في 21، وz في 22.

العنوان	المحتوى	المعنى
10	`LOAD 20`	ACC ← [20]
11	`ADD 21`	ACC ← ACC + [21]
12	`STORE 22`	[22] ← ACC
20	7	x
21	5	y
22	0	z (قبل التنفيذ)

نتابع التعليمة الثانية `ADD 21` خطوة بخطوة (في البداية CO = 11 وACC = 7):

المرحلة	ماذا يحدث؟	CO	RAM	RIM	RI	ACC
الجلب	RAM ← CO	11	11	--	--	7
	قراءة الذاكرة: RIM ← [11]	11	11	`ADD 21`	--	7
	RI ← RIM، و CO ← CO + 1	12	11	`ADD 21`	`ADD 21`	7
فك الترميز	المفكك يفهم: «جمع مع محتوى العنوان 21»	12	11	`ADD 21`	`ADD 21`	7
التنفيذ	RAM ← 21، ثم قراءة: RIM ← [21]	12	21	5	`ADD 21`	7
	UAL: 7 + 5	12	21	5	`ADD 21`	7
الكتابة	ACC ← 12، وتحديث الأعلام	12	21	5	`ADD 21`	**12**

خلاصة

بعد هذه التعليمة: ACC = 12 وCO = 12، فالتعليمة القادمة هي `STORE 22` التي تكتب 12 في الذاكرة، فتصبح z = 12.

هل تعلم؟

المعالجات الحديثة لا تنتظر انتهاء تعليمة لتبدأ الأخرى: بينما تُنفَّذ تعليمة، تُفك ترميز التي بعدها وتُجلب الثالثة. هذا هو **خط الأنابيب** (pipeline) الذي سندرسه في الفصل الرابع.

8.2 تمارين

تمرين 1.2

احسب بالمجمّع الكامل (بتًا بتًا) $0110 + 1011$ على 4 بتات. ما النتيجة وما قيمة CF؟

الحل

البت	A	B	C_{in}	S	C_{out}
0	1	0	0	1	0
1	1	1	0	0	1
2	0	1	1	0	1
3	1	0	1	0	1

النتيجة 0001 و$\text{CF} = 1$: $11 + 6 = 17 = 16 + 1$، و«الـ16» ذهبت إلى CF.

تمرين 2.2

على 8 بتات، أعط قيم AL وCF وZF بعد: (أ) $255 + 1$ (ب) $10 - 10$ (ج) $20 - 10$.

الحل

(أ) $256 - 256 = 0$: $\text{AL} = 0$، $\text{CF} = 1$، $\text{ZF} = 1$.
(ب) $\text{AL} = 0$، $\text{CF} = 0$، $\text{ZF} = 1$.
(ج) -10 $= 20 - 10$، وعلى 8 بتات $246 = \text{F6}_\text{H} = 256 - 10$: $\text{CF} = 1$ (استلاف)، $\text{ZF} = 0$.

تمرين 3.2

حدد نمط العنونة في كل تعليمة: (أ) `mov al, [bx]` (ب) `mov al, 10h` (ج) `mov al, [10h]` (د) `mov al, bl`.

الحل

(أ) غير مباشر بالسجل (ب) فوري (ج) مباشر (د) سجلي.

تمرين 4.2

في المثال السابق ($z = x + y$)، اكتب جدول المراحل للتعليمة الثالثة `STORE 22`.

✔ الحل

الجلب: $12 \leftarrow \text{RAM}$، $\text{STORE } 22 \leftarrow \text{RIM}$، $\text{RI} \leftarrow \text{RIM}$، $\text{CO} = 13$. **فك الترميز:** «اكتب ACC في العنوان 22».
التنفيذ والكتابة: $22 \leftarrow \text{RAM}$، $\text{ACC} \leftarrow \text{RIM}\ (12)$، ثم إشارة كتابة: $12 = [22]$.

الفصل 3

دراسة حالة: المعالج Intel 8086

خلاصة: ماذا سنتعلم في هذا الفصل؟

- عائلة 80x86، والبنية الداخلية للمعالج 8086.
- سجلات المعالج 8086، والعنونة المقطعية (كيف نحسب العنوان الفيزيائي).
- كتابة أول برنامج، وكيف تُنفَّذ تعليمة داخل المعالج.
- التعليمات: النقل، الحساب، المنطق، القفز، الحلقات، ثم كيف تتحول التعليمة إلى لغة الآلة.
- الإدخال والإخراج، المكدس، الإجراءات، والمقاطعات.

1.3 عائلة 80x86

في سنة 1978 أطلقت شركة Intel المعالج 8086. نجح نجاحًا كبيرًا لأن شركة IBM اختارت أخاه 8088 لأول حاسوب شخصي (IBM PC) سنة 1981. ومن يومها، كل معالج جديد من Intel وAMD يبقى قادرًا على تنفيذ برامج 8086. هذه العائلة تسمى x86.

تعريف

التوافق مع الأجيال السابقة (Compatibilité ascendante / Backward compatibility): المعالج الجديد يفهم كل تعليمات المعالجات القديمة من عائلته. برنامج كُتب للمعالج 8086 سنة 1980 يمكن أن يعمل على معالج Core i7 حديث. هذا ما جعل عائلة x86 تبقى أكثر من 45 سنة.

هل تعلم؟

الفرق الوحيد بين 8086 و8088 هو **حافلة المعطيات الخارجية**: 16 بتًا في 8086 و8 بتات فقط في 8088. من الداخل هما متطابقان. اختارت IBM المعالج 8088 لأن الدارات المساعدة ذات 8 بتات كانت أرخص في ذلك الوقت.

المعالج	السنة	عدد البتات	الذاكرة القصوى	الجديد فيه
8086 / 8088	1978	16	1 MB	أول معالج من العائلة
80286	1982	16	16 MB	النمط المحمي
80386	1985	32	4 GB	أول معالج 32 بتًا (IA-32)
80486	1989	32	4 GB	ذاكرة مخبئية ووحدة الأعداد الحقيقية داخله
Pentium	1993	32	4 GB	تنفيذ تعليمتين معًا
(AMD) Athlon 64	2003	64	2^{48} عمليًا	المعمارية AMD64 (x86-64)
Core i7 وما بعده	2008	64	ضخمة	عدة أنوية في الشريحة نفسها

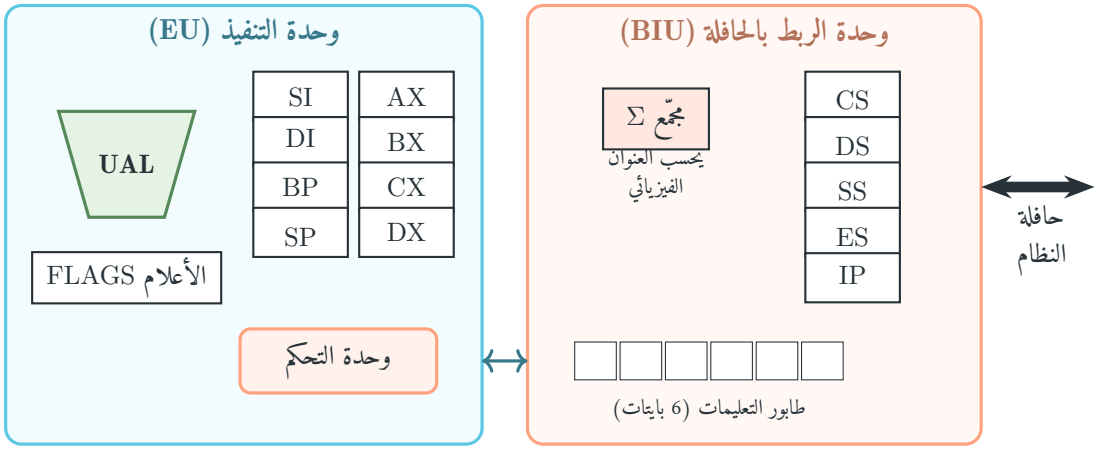
تعريف

الخصائص العامة للمعالج 8086:

- معالج **16** بتًا: سجلاته وحافلة معطياته من 16 بتًا.
- حافلة عناوين من **20** بتًا: يستطيع عنونة 1 MB من الذاكرة.
- حوالي 29 000 ترانزستور، وتردد بين 5 و10 MHz.
- يملك حوالي **100 تعليمة** مختلفة (معالج CISC).

2.3 البنية الداخلية للمعالج 8086

ينقسم المعالج 8086 إلى وحدتين تعملان **في الوقت نفسه**:



- **وحدة الربط بالحافلة** (Bus Interface Unit / BIU): تتكلم مع الذاكرة. تحسب العناوين، تجلب التعليمات مسبقًا وتضعها في طابور من 6 بايتات، وتقرأ وتكتب المعطيات.
- **وحدة التنفيذ** (Execution Unit / EU): تأخذ التعليمات من الطابور، تفك ترميزها، وتنفذها بوحدة الحساب والمنطق.

تشبيه من الحياة اليومية

في مطعم: **النادل** (BIU) يأتي بالطلبات من الزبائن ويضعها في صف أمام الطباخ، بينما **الطباخ** (EU) يحضّر الطلب الحالي. الطباخ لا ينتظر النادل، لأن الطلب القادم موجود دائمًا في الصف. هذه أول فكرة لخط الأنابيب!

3.3 سجلات المعالج 8086

يملك المعالج 8086 **14 سجلًا** من 16 بتًا.

1.3.3 السجلات العامة

أربعة سجلات للمعطيات، ويمكن تقسيم كل واحد منها إلى نصفين من 8 بتات:

	البتات 15 إلى 8	البتات 7 إلى 0	
المرِكّم: الحساب، الضرب، القسمة، الإدخال والإخراج	AH	AL	AX
القاعدة: يحفظ عنوانًا في الذاكرة	BH	BL	BX
العدّاد: عدد دورات الحلقات	CH	CL	CX
المعطيات: مع AX في الضرب والقسمة	DH	DL	DX

مثال

إذا كان $AX = 1234_H$ فإن $AH = 12_H$ و$AL = 34_H$. إذا غيّرنا AL إلى FF_H يصبح $AX = 12FF_H$.

لماذا نقسم السجلات؟ لأننا نعمل كثيرًا بقيم صغيرة (حرف، رقم بين 0 و255). بدل أن نضيّع سجلًا كاملًا من 16 بتًا لحرف واحد، نستطيع وضع حرفين في سجل واحد: واحد في AH وآخر في AL.

انتبه

هذه السجلات «عامة» لأننا نستطيع استعمالها في أي حساب، لكن بعض التعليمات تفرض سجلًا معينًا: `loop` تستعمل دائمًا CX، و`mul` و`div` تستعملان دائمًا AX وDX، ووظائف `int 21h` تنتظر رقم الوظيفة في AH. لهذا يجب أن نحفظ هذه الأدوار.

2.3.3 سجلات المؤشرات والفهارس

السجل	الاسم	الدور
SI	Source Index	فهرس المصدر (للجداول والسلاسل)
DI	Destination Index	فهرس الوجهة
SP	Stack Pointer	يشير إلى قمة المكدس
BP	Base Pointer	للوصول إلى معطيات داخل المكدس

مثال

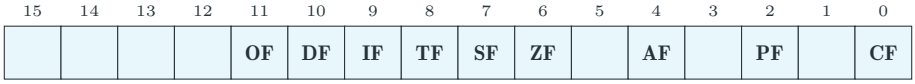
لنمر على جدول من 10 عناصر، نضع عنوان بدايته في SI، ثم نزيد SI في كل دورة (`inc si`): في الدورة الأولى يشير SI إلى العنصر الأول، وفي الثانية إلى الثاني، وهكذا. SI هنا يلعب دور **الإصبع** الذي نضعه على العنصر الذي نقرؤه الآن.

3.3.3 سجلات المقاطع

السجل	الاسم	يشير إلى بداية...
CS	Code Segment	مقطع البرنامج (التعليمات)
DS	Data Segment	مقطع المعطيات (المتغيرات)
SS	Stack Segment	مقطع المكدس
ES	Extra Segment	مقطع إضافي (نسخ السلاسل مثلًا)

4.3.3 مؤشر التعليمة وسجل الأعلام

- **IP** (Instruction Pointer): عنوان التعليمة القادمة داخل مقطع البرنامج. هو CO الذي رأيناه في الفصل الثاني. لا يمكن تغييره مباشرة بـ`mov`، بل بالقفز والاستدعاء.
- **FLAGS**: 9 بتات مستعملة من أصل 16:



العلم	المعنى	العلم	المعنى
CF	حمل أو استلاف خارج من آخر بت	OF	فيضان: نتيجة خاطئة للأعداد بإشارة
PF	عدد الآحاد في البايت الأول زوجي	DF	اتجاه السلاسل (0: تصاعدي)
AF	حمل من البت 3 إلى البت 4	IF	السماح بالمقاطعات العتادية
ZF	النتيجة صفر	TF	التنفيذ خطوة بخطوة (للتصحيح)
SF	النتيجة سالبة (البت الأخير 1)		

4.3 العنونة المقطعية

1.4.3 المشكلة

سجلات 8086 من 16 بتًا، فهي تستطيع كتابة عناوين من 0 إلى $65535 = 2^{16} - 1$ فقط (أي 64 KB). لكن حافلة العناوين من 20 بتًا (1 MB). كيف نكتب عنوانًا من 20 بتًا بسجلات من 16 بتًا؟

2.4.3 الحل: المقطع والإزاحة

نستعمل **سجلين**: سجل **المقطع** يحدد بداية منطقة من الذاكرة، و**الإزاحة** (offset) تحدد المسافة داخل هذه المنطقة. نكتب العنوان المنطقي هكذا: segment:offset.

تعريف

$$\text{العنوان الفيزيائي} = \text{المقطع} \times 16 + \text{الإزاحة}$$

الضرب في 16 = إضافة صفر واحد على اليمين في الست عشري.

خطوة بخطوة: العنوان الفيزيائي لـ $CS = 1000H$ و$IP = 0234H$

1) نضرب المقطع في 16 (نضيف صفرًا): 1000_H يصبح 10000_H.
2) نضيف الإزاحة:

$$\begin{array}{r} 10000 \\ 0234 \; + \\ \hline 10234 \end{array}$$

3) العنوان الفيزيائي هو 10234_H، وهو عنوان من 20 بتًا (5 أرقام ست عشرية).

3.4.3 لماذا نضرب في 16؟

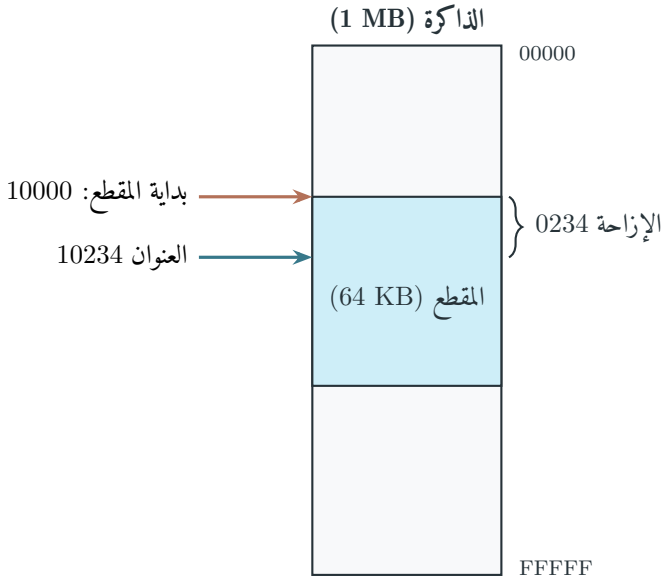
لو استعملنا المقطع كما هو (دون ضرب)، لما تجاوزنا العنوان $FFFF_H + FFFF_H = 1FFFE_H$، أي حوالي 128 KB فقط. بالضرب في 16، يصبح المقطع قادرًا على الإشارة إلى أي مكان في 1 MB: $FFFF_H \times 16 = FFFF0_H$. ولهذا يبدأ كل مقطع دائمًا عند عنوان يقبل القسمة على 16، ويسمى هذا الجزء من 16 بايتًا **فقرة** (Paragraphe / Paragraph).

خطوة بخطوة: ما المنطقة التي يغطيها المقطع $DS = 2000H$؟

1) أصغر إزاحة هي 0000_H: $20000_H + 0000_H = 20000_H$.
2) أكبر إزاحة هي $FFFF_H$: $20000_H + FFFF_H = 2FFFF_H$.
3) المقطع يغطي المنطقة من 20000_H إلى $2FFFF_H$، أي $2^{16} = 65536$ بايتًا = 64 KB.

تشبيه من الحياة اليومية

تخيّل مدينة فيها شوارع كثيرة. العنوان الكامل لبيت هو «**الشارع** رقم 12، **المنزل** رقم 34». سجل المقطع هو رقم الشارع (يحدد بداية منطقة)، والإزاحة هي رقم المنزل داخل الشارع. ساعي البريد يحتاج المعلومتين معًا ليجد البيت.



انتبه

عنوان فيزيائي واحد يمكن كتابته بعدة طرق: 1000:0234 و1023:0004 يعطيان العنوان نفسه 10234_H. تحقق بنفسك: $10230_H + 0004_H = 10234_H$.

4.4.3 الأزواج الافتراضية

كل سجل إزاحة يُستعمل افتراضيًا مع سجل مقطع معين:

الإزاحة	المقطع الافتراضي	الاستعمال
IP	CS	جلب التعليمات
SP، BP	SS	المكدس
BX، SI، عنوان مباشر	DS	المتغيرات
DI (في تعليمات السلاسل)	ES	وجهة النسخ

هل تعلم؟

في برامج COM. التي نكتبها في هذا الفصل، تكون المقاطع الأربعة CS = DS = SS = ES متساوية: البرنامج كله داخل مقطع واحد من 64 KB، ويبدأ عند الإزاحة 100_H. هذا معنى السطر `org 100h` الذي سنجده في كل برامجنا.

5.3 أنماط العنونة في 8086

تعريف

العنوان الفعلي (Adresse effective, EA / Effective Address): الإزاحة التي يحسبها المعالج لمعامل في الذاكرة. ثم يحسب العنوان الفيزيائي $\text{EA} \times 16 + \text{DS} =$.

النمط	مثال	العنوان الفعلي EA
فوري	`mov ax, 5`	لا يوجد: القيمة في التعليمة
سجلي	`mov ax, bx`	لا يوجد: القيمة في سجل
مباشر	`mov ax, [0200h]`	$\text{EA} = 0200_\text{H}$
غير مباشر بسجل	`mov ax, [bx]`	$\text{EA} = \text{BX}$ (أو SI، DI، BP)
قاعدي	`mov ax, [bx+4]`	$\text{EA} = \text{BX} + 4$
مفهرس	`mov ax, [si+4]`	$\text{EA} = \text{SI} + 4$
قاعدي مفهرس	`mov ax, [bx+si+4]`	$\text{EA} = \text{BX} + \text{SI} + 4$

خطوة بخطوة: أين يقرأ `mov ax, [bx+si+4]`؟

نفترض: $\text{DS} = 2000_\text{H}$، $\text{BX} = 0100_\text{H}$، $\text{SI} = 0020_\text{H}$.

1) العنوان الفعلي: $\text{EA} = 0100_\text{H} + 0020_\text{H} + 4 = 0124_\text{H}$.

2) العنوان الفيزيائي: $2000_\text{H} \times 16 + 0124_\text{H} = 20000_\text{H} + 0124_\text{H} = 20124_\text{H}$.

3) يقرأ المعالج كلمة (بايتين): البايت في 20124_H يذهب إلى AL، والبايت في 20125_H يذهب إلى AH.

خطوة بخطوة: `mov ax, [bp+2]` يستعمل المقطع SS

نفترض $\text{SS} = 3000_\text{H}$ و$\text{BP} = \text{FFF0}_\text{H}$.

1) العنوان الفعلي: $\text{EA} = \text{FFF0}_\text{H} + 2 = \text{FFF2}_\text{H}$.

2) بما أننا استعملنا BP، فالمقطع الافتراضي هو SS وليس DS.

3) العنوان الفيزيائي: $30000_\text{H} + \text{FFF2}_\text{H} = 3\text{FFF2}_\text{H}$.

هذا مهم في الإجراءات: BP يُستعمل لقراءة المعاملات الموجودة في المكدس.

انتبه

المعالج 8086 يحفظ الكلمة في الذاكرة بترتيب **البايت الصغير أولًا** (little endian): الكلمة 1234_H تُخزن 34 ثم 12. لا يُسمح أيضًا بمعاملين في الذاكرة في التعليمة نفسها: `mov [a], [b]` خطأ.

6.3 أول برنامج

1.6.3 الأدوات

نكتب البرنامج في **EMU8086**: New → COM template File، ثم نضغط Emulate وننفذ تعليمة بتعليمة بالزر Single step.
يمر كل برنامج بلغة التجميع بثلاث مراحل:



1) **الكتابة**: نكتب التعليمات في ملف نصي (المصدر).
2) **التجميع**: المجمِّع يحوّل كل تعليمة إلى بايتات لغة الآلة، ويحسب عناوين المتغيرات والتسميات.
3) **التحميل والتنفيذ**: النظام ينسخ البايتات في الذاكرة ابتداءً من الإزاحة 100_{H}، ويضع $\text{IP} = 100_{\text{H}}$، فيبدأ المعالج التنفيذ.

2.6.3 البرنامج

```
1 ; My first 8086 program: print a message on the screen
2 org 100h              ; .COM program: code starts at offset 100h
3
4     lea dx, msg       ; DX = address of the message
5     mov ah, 09h       ; DOS function 09h = print a string ending with '$'
6     int 21h           ; call DOS
7     ret               ; end of the program, back to DOS
8
9 msg db 'Salam, I am the 8086!$'
```

screen

```
Salam, I am the 8086!
```

السطر	ماذا يفعل؟
1	تعليق: كل ما بعد ; لا ينفذه المعالج، هو شرح للقارئ.
2	org 100h: البرنامج من نوع COM. يبدأ عند الإزاحة 100_{H}.
4	نضع في DX عنوان الرسالة msg.
5	نختار وظيفة نظام DOS رقم 09_{H}: «اكتب السلسلة الموجودة في DX».
6	int 21h: نطلب من DOS تنفيذ الوظيفة المختارة في AH.
7	ret: نهاية البرنامج والعودة إلى النظام.
9	نعرّف المتغير msg بـ db (define byte). العلامة $ تعني نهاية النص.

انتبه

نضع المعطيات **بعد** ret. لو وضعناها قبل التعليمات، لحاول المعالج تنفيذ حروف الرسالة كأنها تعليمات!

خطوة بخطوة: البرنامج في الذاكرة بعد التحميل

هذه هي البايتات الحقيقية التي أنتجها المجمِّع:

الإزاحة	البايتات	المحتوى
0100	8D 16 09 01	`lea dx, msg` (العنوان 0109_H)
0104	B4 09	`mov ah, 09h`
0106	CD 21	`int 21h`
0108	C3	`ret`
0109	53 61 6C 61 6D …	الحروف `S a l a m` …

البرنامج كله 9 بايتات من التعليمات، والرسالة تبدأ مباشرة بعدها في 0109_H. لاحظ أن المجمِّع حسب عنوان `msg` بنفسه ووضعه داخل تعليمة `lea`.

3.6.3 التعريف بالمتغيرات

الكتابة	المعنى
`x db 5`	بايت واحد قيمته 5
`y dw 1234h`	كلمة (بايتان) قيمتها 1234_H
`t db 14, 3, 27`	جدول من 3 بايتات
`buf db 10 dup(0)`	10 بايتات كلها 0
`s db 'Hi$'`	سلسلة حروف

7.3 مراحل تنفيذ تعليمة داخل 8086

لنتابع التعليمة `mov cx, [si]` من البرنامج `transfer.asm` (القسم الآتي). في الذاكرة، هذه التعليمة مكتوبة بالبايتين 8B 0C في الإزاحة $010C_H$، و$SI = 0112_H$.

المرحلة	الوحدة	ما يحدث
1	BIU	تحسب العنوان الفيزيائي $CS \times 16 + IP$، وتجلب البايتين 8B 0C إلى الطابور (غالبًا قبل أن نحتاجهما).
2	EU	تأخذ 8B من الطابور: «نقل كلمة من الذاكرة إلى سجل». ثم 0C: «السجل CX، والعنوان في SI». IP يصبح $010E_H$.
3	EU	تحسب العنوان الفعلي: $EA = SI = 0112_H$.
4	BIU	تحسب $DS \times 16 + 0112_H$، ترسله على حافلة العناوين، مع إشارة قراءة.
5	BIU	تصل الكلمة $ABCD_H$ على حافلة المعطيات (CD ثم AB).
6	EU	تكتب النتيجة في CX: $CX = ABCD_H$. الأعلام لا تتغير مع `mov`.

8.3 مجموعة التعليمات بالتفصيل

نستعمل هذه الرموز في الجداول: reg = سجل، mem = خانة ذاكرة، imm = قيمة فورية.

1.8.3 تعليمات النقل

التعليمة	المعنى	مثال
`mov dest, src`	dest ← src	`mov ax, bx`
`xchg a, b`	تبادل القيمتين	`xchg ax, bx`
`lea reg, mem`	reg يأخذ عنوان المتغير	`lea si, tab`
`pop dest / push src`	وضع في المكدس / أخذ منه	`push ax`

```
; Data transfer: MOV, XCHG, LEA
org 100h

    mov ax, 1234h     ; AX = 1234h (immediate)
    mov bl, ah        ; BL = high byte of AX = 12h
    mov bh, al        ; BH = low byte of AX  = 34h
    xchg ax, bx       ; swap AX and BX
    lea si, num       ; SI = address of num
    mov cx, [si]      ; CX = the word stored at num
    mov [res], ax     ; store AX in memory (variable res)
    ret

num dw 0ABCDh
res dw 0
```

السطر	التعليمة	AX	BX	CX	SI
4	`mov ax, 1234h`	1234	0000	0000	0000
5	`mov bl, ah`	1234	0012	0000	0000
6	`mov bh, al`	1234	3412	0000	0000
7	`xchg ax, bx`	3412	1234	0000	0000
8	`lea si, num`	3412	1234	0000	0112
9	`mov cx, [si]`	3412	1234	ABCD	0112
10	`mov [res], ax`	3412	1234	ABCD	0112

السطر	ماذا يفعل؟
5–6	نسخ نصفي AX مقلوبين في BX: BH = 34 وBL = 12.
8	`lea` تعطي **العنوان** 0112_H وليس القيمة.
9	نقرأ الكلمة الموجودة في العنوان 0112_H: $ABCD_H$.
10	السطر الوحيد الذي يكتب في الذاكرة: `res` تصبح 3412_H.

2.8.3 التعليمات الحسابية

التعليمة	المعنى	الأعلام
`add d, s`	$d \leftarrow d + s$	كلها
`adc d, s`	$d \leftarrow d + s + \text{CF}$ (لجمع أعداد كبيرة)	كلها
`sub d, s`	$d \leftarrow d - s$	كلها
`inc d`	$d \leftarrow d + 1$	كلها إلا CF
`dec d`	$d \leftarrow d - 1$	كلها إلا CF
`neg d`	$d \leftarrow -d$ (المتمم لاثنين)	كلها
`cmp a, b`	يحسب $a - b$ **دون حفظ النتيجة**	كلها
`mul s`	ضرب دون إشارة (`imul` بإشارة)	OF ،CF
`div s`	قسمة دون إشارة (`idiv` بإشارة)	غير محددة

```
; Arithmetic: ADD, SUB, INC, DEC, NEG
org 100h

    mov al, 25        ; AL = 25
    add al, 17        ; AL = 25 + 17 = 42
    sub al, 2         ; AL = 42 - 2  = 40
    inc al            ; AL = 41
    dec al            ; AL = 40
    neg al            ; AL = -40 (two's complement)
    mov bx, 1000      ; BX = 1000
    add bx, 2500      ; BX = 3500
    ret
```

السطر	التعليمة	النتيجة	بالعشري	CF
4	`mov al, 25`	AL = 19	25	0
5	`add al, 17`	AL = 2A	42	0
6	`sub al, 2`	AL = 28	40	0
7	`inc al`	AL = 29	41	0
8	`dec al`	AL = 28	40	0
9	`neg al`	AL = D8	−40	1
10	`mov bx, 1000`	BX = 03E8	1000	1
11	`add bx, 2500`	BX = 0DAC	3500	0

خطوة بخطوة: لماذا -40 يُكتب $D8_H$؟ (المتمم لإثنين)

1) نكتب 40 بالثنائي: $0010\,1000_2$.
2) نقلب كل البتات: $1101\,0111_2$.
3) نضيف 1: $1101\,1000_2 = D8_H$.

تحقق: $28_H + D8_H = 100_H$، $40 + (-40) = 0$، وعلى 8 بتات يبقى 00. صحيح!

الضرب والقسمة

المعالج يختار المكان حسب حجم المعامل:

التعليمة	المعامل	العملية	النتيجة
`mul bl`	8 بتات	AL × BL	AX
`mul cx`	16 بتًا	AX × CX	DX:AX (32 بتًا)
`div bl`	8 بتات	AX ÷ BL	AL = الحاصل، AH = الباقي
`div cx`	16 بتًا	DX:AX ÷ CX	AX = الحاصل، DX = الباقي

```
1  ; Multiplication and division
2  org 100h
3
4      mov al, 12        ; first number
5      mov bl, 11        ; second number
6      mul bl            ; AX = AL * BL = 132
7
8      mov ax, 300       ; AX = 300
9      mov cx, 500
10     mul cx            ; DX:AX = AX * CX = 150000
11
12     mov ax, 47        ; dividend
13     mov bl, 5         ; divisor
14     div bl            ; AL = 47 / 5 = 9 , AH = remainder = 2
15     ret
```

السطر	ماذا يفعل؟
6	$12 \times 11 = 132 = 84_H$: $AX = 0084_H$.
10	$300 \times 500 = 150000$، وهو أكبر من 65535، فيتوزع على سجلين: $DX = 0002_H$ و$AX = 49F0_H$ (لأن $249F0_H = 150000$).
14	$47 = 5 \times 9 + 2$: $AL = 09$ (الحاصل) و$AH = 02$ (الباقي)، أي $AX = 0209_H$.

انتبه

قبل `div cx` يجب وضع $DX = 0$ إذا كان العدد المقسوم في AX فقط، وإلا اعتبر المعالج DX جزءًا من العدد. والقسمة على 0 توقف البرنامج (مقاطعة رقم 0).

جمع أعداد أكبر من 16 بتًا: adc

المعالج 8086 يجمع 16 بتًا في المرة الواحدة. لجمع عددين من 32 بتًا، نقسم كل عدد إلى كلمتين (عالية ومنخفضة)، ونجمع الكلمتين المنخفضتين بـadd، ثم العاليتين بـadc التي تضيف الحمل CF القادم من الجمع الأول، تمامًا كما نحمل «واحدًا» إلى العمود التالي عند الجمع بالقلم.

```
; Add two 32-bit numbers with a 16-bit processor
; 0001FFFFh + 00000001h = 00020000h
org 100h

    mov ax, 0FFFFh    ; low word of the 1st number
    mov dx, 0001h     ; high word of the 1st number
    add ax, 0001h     ; low words: FFFF + 1 = 0000, carry CF = 1
    adc dx, 0000h     ; high words: 0001 + 0 + CF = 0002
    ret               ; result DX:AX = 0002:0000
```

السطر	التعليمة	DX	AX	CF
5–6	تحميل العدد الأول	0001	FFFF	0
7	add ax, 0001h	0001	0000	1
8	adc dx, 0000h	0002	0000	0

النتيجة DX:AX = 0002:0000 = 00020000_H. لو استعملنا add في السطر 8 لضاع الحمل، ولحصلنا على 00010000_H، وهي نتيجة خاطئة.

3.8.3 التعليمات المنطقية

التعليمة	الاستعمال المعتاد	مثال
and	إطفاء بتات (القناع)	and al, 0Fh
or	إشعال بتات	or al, 80h
xor	قلب بتات، أو تصفير سجل	xor cx, cx
not	قلب كل البتات	not al
test	and دون حفظ النتيجة (فحص بت)	test al, 1
shr / shl	إزاحة: ضرب أو قسمة على 2	shl bl, 1
ror / rol	تدوير: البت الخارج يعود من الجهة الأخرى	rol al, 1

```
1  ; Logic and shift instructions
2  org 100h
3
4      mov al, 10110110b ; AL = B6h
5      and al, 00001111b ; keep only the 4 low bits  -> 06h
6      or  al, 10000000b ; set bit 7 to 1            -> 86h
7      xor al, 11111111b ; invert all the bits       -> 79h
8      not al            ; invert again              -> 86h
9      mov bl, 5
10     shl bl, 1         ; shift left  = multiply by 2 -> 10
11     shl bl, 1         ;                              -> 20
12     shr bl, 1         ; shift right = divide by 2   -> 10
13     xor cx, cx        ; quick way to put CX = 0
14     ret
```

السطر	التعليمة	AL بالثنائي	ست عشري
4	`mov al, 10110110b`	1011 0110	B6
5	`and al, 00001111b`	0000 0110	06
6	`or al, 10000000b`	1000 0110	86
7	`xor al, 11111111b`	0111 1001	79
8	`not al`	1000 0110	86

السطر	ماذا يفعل؟
5	`and` مع 1 يبقي البت، ومع 0 يطفئه: نحتفظ بالبتات الأربعة الأولى فقط.
6	`or` مع 1 يشعل البت: البت 7 أصبح 1.
10–12	BL: 10 → 20 → 10 → 5. الإزاحة إلى اليسار تضرب في 2، وإلى اليمين تقسم على 2.
13	`xor cx, cx`: كل بت مع نفسه يعطي 0، إذن CX = 0. أسرع وأقصر من `mov cx, 0`.

استعمالات عملية للأقنعة

خطوة بخطوة: هل العدد في AL زوجي؟

العدد زوجي إذا كان البت 0 يساوي 0. نكتب `test al, 1`: هذه التعليمة تحسب AL · 1 دون أن تغيّر AL.

- إذا كان AL = 6 = $0000\,0110_2$: البت 0 يساوي 0، النتيجة 0، و ZF = 1: العدد زوجي.
- إذا كان AL = 7 = $0000\,0111_2$: البت 0 يساوي 1، النتيجة 1، و ZF = 0: العدد فردي.

بعدها نكتب `jz zawji` للقفز في حالة العدد الزوجي.

خطوة بخطوة: تحويل حرف صغير إلى كبير بالقناع

في جدول ASCII: $'a' = 61_H = 0110\,0001_2$ و $'A' = 41_H = 0100\,0001_2$. الفرق الوحيد هو البت 5. إذن `and al, 11011111b` (أي `0DFh`) يطفئ البت 5 فيحوّل أي حرف صغير إلى كبير، ويترك الحرف الكبير كما هو. هذه طريقة أخرى غير `sub al, 32` التي سنراها في قسم الإدخال.

4.8.3 التحكم في التدفق: القفزات

عادة ينفذ المعالج التعليمات بالترتيب. **القفز** يغيّر IP فيذهب التنفيذ إلى مكان آخر.

التعليمة	تقفز إذا...	الأعلام
`jmp L`	دائمًا (قفز غير مشروط)	--
`jz` / `je`	متساويان / النتيجة صفر	ZF = 1
`jnz` / `jne`	غير متساويين	ZF = 0
`jge` / `jg`	أكبر / أكبر أو يساوي (بإشارة)	OF ،SF ،ZF
`jle` / `jl`	أصغر / أصغر أو يساوي (بإشارة)	OF ،SF
`jae` / `ja`	أكبر / أكبر أو يساوي (دون إشارة)	ZF ،CF
`jbe` / `jb`	أصغر / أصغر أو يساوي (دون إشارة)	ZF ،CF

تشبيه من الحياة اليومية

`cmp` يشبه **وضع شيئين على ميزان**، والقفز المشروط يشبه قولك: «إذا مالت الكفة اليمنى فاذهب إلى اليمين». الميزان (`cmp`) يضبط الأعلام، والقفز ينظر إليها.

مثال: إذا ...وإلا

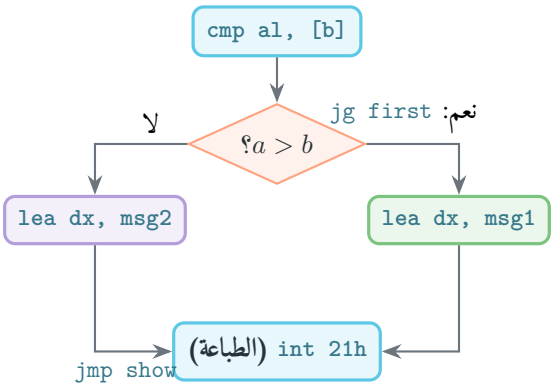
```
; IF ... ELSE : print which number is bigger
org 100h

    mov al, [a]       ; AL = a
    cmp al, [b]       ; compare a with b (computes a - b)
    jg  first         ; jump if a > b (signed)
    lea dx, msg2      ; else: b is bigger or equal
    jmp show
first:
    lea dx, msg1      ; a is bigger
show:
    mov ah, 09h       ; print the chosen message
    int 21h
    ret

a    db 7
b    db 12
msg1 db 'a is bigger$'
msg2 db 'b is bigger or equal$'
```

screen

```
b is bigger or equal
```



السطر	ماذا يفعل؟
5	`cmp` يحسب $5 - 12 = -7$ ولا يحفظه، لكنه يضبط الأعلام ($\mathrm{SF} = 1$).
6	`jg`: هل $a > b$؟ لا، فلا نقفز ونكمل في السطر 7.
7–8	نختار الرسالة الثانية، ثم `jmp show` **ضروري** لكي لا ننفذ السطر 10.
12–13	الطباعة مشتركة بين الحالتين.

انتبه

نسي كثيرًا `jmp show` في السطر 8! بدونه ينفذ البرنامج `lea dx, msg1` أيضًا، فيطبع دائمًا الرسالة الأولى.

بإشارة أم دون إشارة؟ `jg` أو `ja`

البتات نفسها يمكن قراءتها بطريقتين (رأينا ذلك في الفصل الثاني). $\mathrm{AL} = \mathrm{C8_H}$ يساوي 200 دون إشارة، ويساوي -56 بإشارة. لذلك يملك المعالج نوعين من القفزات: `jb`/`ja` للأعداد دون إشارة، و`jl`/`jg` للأعداد بإشارة.

```
; Same bits, two meanings: 200 is also -56
org 100h

    mov al, 200       ; AL = C8h
    cmp al, 100
    ja  bigger_u      ; unsigned: 200 > 100 ?
    lea dx, no_u
    jmp test_s
bigger_u:
    lea dx, yes_u
test_s:
    mov ah, 09h
    int 21h
    mov al, 200
    cmp al, 100
    jg  bigger_s      ; signed: -56 > 100 ?
    lea dx, no_s
    jmp fin
bigger_s:
    lea dx, yes_s
fin:
    mov ah, 09h
    int 21h
    ret

yes_u db 'unsigned: 200 > 100', 13, 10, '$'
no_u  db 'unsigned: 200 <= 100', 13, 10, '$'
yes_s db 'signed: -56 > 100$'
no_s  db 'signed: -56 <= 100$'
```

screen

```
unsigned: 200 > 100
signed: -56 <= 100
```

السطر	ماذا يفعل؟
4–6	`ja`: نقارن 200 مع 100 كأعداد موجبة. $200 > 100$، فنقفز.
14–16	`jg`: البتات نفسها $C8_H$، لكن نقرؤها بإشارة: $-56 > 100$؟ لا، فلا نقفز.

انتبه

اختر القفزة حسب نوع **المعطيات**: الأعمار، العناوين، رموز الحروف، الأحجام: دون إشارة (`ja`، `jb`). درجات الحرارة، الأرصدة، الفروق: بإشارة (`jg`، `jl`).

من الخوارزمية إلى لغة التجميع

كل بنية تحكم تعلمناها في مقياس الخوارزميات لها «ترجمة» ثابتة في لغة التجميع. الفكرة دائمًا هي نفسها: **نقارن** بـ`cmp`، ثم **نقفز** حسب النتيجة.

البنية	بلغة C	بلغة التجميع

إذا	`if (a == b) x = 1;`	`cmp a, b`
		`jne end`
		`mov x, 1`
		`end:`

طالما	`while (cx > 0) { ... }`	`L: cmp cx, 0`
		`jle end`
		`...`
		`jmp L`
		`end:`

من أجل	`for (cx = 10; cx > 0; cx--)`	`mov cx, 10`
		`L: ...`
		`loop L`

انتبه

لاحظ في ترجمة `if`: نقفز بالشرط **المعاكس** (`jne` بدل `je`). نقول للمعالج: «إذا لم يكونا متساويين، تجاوز تعليمة الإسناد». هذه الطريقة تجعل البرنامج أقصر.

5.8.3 الحلقات والتكرار

تعريف

`loop L` تساوي تعليمتين: `dec cx` ثم «إذا كان $CX \neq 0$ فاقفز إلى L». إذن CX هو عدّاد الحلقة.

مجموع الأعداد من 1 إلى 10

```
; FOR loop with LOOP: sum = 1 + 2 + ... + 10
org 100h

    mov ax, 0          ; sum = 0
    mov cx, 10         ; CX = number of turns (and current value)
again:
    add ax, cx         ; sum = sum + CX
    loop again         ; CX = CX - 1 ; if CX <> 0 jump to again
    mov [sum], ax      ; sum = 55
    ret

sum dw 0
```

الدورة	1	2	3	4	5	6	7	8	9	10
CX قبل add	10	9	8	7	6	5	4	3	2	1
AX بعد add	10	19	27	34	40	45	49	52	54	55

بعد الدورة العاشرة يصبح $CX = 0$ فتتوقف الحلقة، و$AX = 0037_H = 55$. لاحظ أننا نجمع بالعكس: $1 + \cdots + 9 + 10$، والنتيجة نفسها.

انتبه

إذا دخلنا الحلقة و$CX = 0$، فإن `loop` تنقص CX أولًا فيصبح $FFFF_H$، ثم تكرر 65535 مرة أخرى! لتجنب ذلك نضع قبل الحلقة `jcxz fin`: «إذا كان $CX = 0$ فاقفز إلى النهاية».

البحث عن أكبر قيمة في جدول

```
; WHILE loop: find the biggest value in an array
org 100h

    lea si, tab       ; SI points to the first element
    mov cx, 6         ; 6 elements
    mov al, [si]      ; max = first element
next:
    cmp [si], al      ; compare current element with max
    jbe skip          ; if element <= max, keep max
    mov al, [si]      ; else max = element
skip:
    inc si            ; go to the next element
    loop next
    mov [max], al
    ret

tab db 14, 3, 27, 9, 31, 8
max db 0
```

SI يشير إلى	العنصر	العنصر > AL؟	AL بعدها	CX بعد loop
tab[0]	14	لا (يساويه)	14	5
tab[1]	3	لا	14	4
tab[2]	27	نعم	27	3
tab[3]	9	لا	27	2
tab[4]	31	نعم	31	1
tab[5]	8	لا	31	0

النتيجة: $AL = 1F_H = 31$، وتُحفظ في `max`. استعملنا `jbe` لأن القيم **دون إشارة**.

التكرار بالبادئة rep

تعليمات السلاسل تعمل على DS:SI (المصدر) وES:DI (الوجهة)، والبادئة rep تكررها CX مرة.

```
; Repetition with REP: copy a string with one instruction
org 100h

    lea si, src       ; SI = source
    lea di, dst       ; DI = destination
    mov cx, 6         ; number of bytes to copy
    cld               ; direction = forward (SI and DI increase)
    rep movsb         ; repeat CX times: [DI] = [SI], SI++, DI++
    lea dx, dst
    mov ah, 09h       ; print the copy
    int 21h
    ret

src db 'Hello$'
dst db 6 dup(0)
```

screen

```
Hello
```

السطر	ماذا يفعل؟
7	cld: نجعل DF = 0، فيزيد SI وDI بعد كل نسخ.
8	rep movsb تنسخ بايتًا من [SI] إلى [DI]، وتكرر 6 مرات: الحروف الخمسة و$.

6.8.3 توليد لغة الآلة

المجمِّع يحوّل كل تعليمة إلى بايتات. هذه البايتات الحقيقية للبرنامج machine.asm:

```
; Small program to study its machine code
org 100h

    mov ax, 1234h
    mov bx, ax
    add al, 5
    mov [bx], al
    inc cx
    jmp fin
    nop
fin:
    ret
```

العنوان	لغة الآلة (ست عشري)	التعليمة
0100	B8 34 12	`mov ax, 1234h`
0103	89 C3	`mov bx, ax`
0105	04 05	`add al, 5`
0107	88 07	`mov [bx], al`
0109	41	`inc cx`
010A	EB 01	`jmp fin`
010C	90	`nop`
010D	C3	`ret`

خطوة بخطوة: فك ترميز B8 34 12

1) B8 = 1011 1000$_2$: الخمسة بتات الأولى 10111 تعني «`mov` قيمة فورية في سجل 16 بتًا»، والبتات الثلاثة الأخيرة 000 = رقم السجل AX.
2) 34 12: القيمة 1234$_H$ مكتوبة بالبايت الصغير أولًا (little endian).

خطوة بخطوة: فك ترميز 89 C3 (`mov bx, ax`)

البايت الأول 89 = 1 0 0010 0 1$_2$:

- 100010: الرمز العملي لـ`mov` بين سجل وسجل أو ذاكرة.
- d = 0: السجل المذكور في الحقل reg هو **المصدر**.
- w = 1: نعمل على 16 بتًا (لو كان 0 لعملنا على 8 بتات).

البايت الثاني C3 = 11 000 011$_2$، ويسمى ModR/M:

11	000	011
mod: سجل	AX = reg	BX = r/m

إذن: انقل AX (المصدر) إلى BX.

الرقم	000	001	010	011	100	101	110	111
w = 1	AX	CX	DX	BX	SP	BP	SI	DI
w = 0	AL	CL	DL	BL	AH	CH	DH	BH

مثال: EB 01 و88 07

88 = 1 0 0010 0 0$_2$: `mov` على 8 بتات. 07 = 00 000 111$_2$: mod = 00 (ذاكرة دون إزاحة)، reg = 000 (AL)، r/m = 111 ([BX]). إذن `mov [bx], al`.

EB 01: قفز قصير، والإزاحة نسبية: بعد قراءة `jmp` يكون IP = 010C$_H$، ونضيف 1 فنصل إلى 010D$_H$ (`fin`). هكذا قفزنا فوق `nop`.

9.3 إدارة الإدخال والإخراج

1.9.3 المنافذ: `in` و`out`

يتصل المعالج بالأجهزة عبر **منافذ** (Ports / I/O ports) مرقمة من 0 إلى $FFFF_H$.

- `in al, 60h`: اقرأ بايتًا من المنفذ 60_H (لوحة المفاتيح في الحاسوب الشخصي).
- `out 61h, al`: أرسل AL إلى المنفذ 61_H (مكبر الصوت).

البرامج العادية نادرًا ما تستعمل المنافذ مباشرة، بل تطلب ذلك من نظام التشغيل عبر المقاطعات.

2.9.3 خدمات DOS: المقاطعة `int 21h`

AH	الوظيفة	المدخلات والمخرجات
01_H	قراءة حرف من لوحة المفاتيح مع إظهاره	المخرج: الحرف في AL
02_H	كتابة حرف	المدخل: الحرف في DL
09_H	كتابة سلسلة تنتهي بـ$	المدخل: عنوانها في DX
$4C_H$	إنهاء البرنامج	رمز النهاية = AL

```
; Read a lowercase letter and print it in uppercase
org 100h

    lea dx, ask
    mov ah, 09h       ; print the question
    int 21h
    mov ah, 01h       ; DOS: read one key -> AL (with echo)
    int 21h
    sub al, 32        ; 'a' (97) - 32 = 'A' (65)
    mov bl, al        ; keep the letter in BL
    lea dx, ans
    mov ah, 09h
    int 21h
    mov dl, bl        ; DOS function 02h prints the character in DL
    mov ah, 02h
    int 21h
    ret

ask db 'Type a letter: $'
ans db 13, 10, 'Uppercase: $'
```

screen

```
Type a letter: m
Uppercase: M
```

السطر	ماذا يفعل؟
7–8	ننتظر أن يضغط المستعمل حرفًا (هنا `m`)، فيصل رمزه ASCII في AL: 109.
9	في جدول ASCII، الحرف الكبير أصغر من الصغير بـ32: 77 = 109 − 32، وهو رمز `'M'`.
10	نحفظ الحرف في BL، لأن الطباعة في السطر 13 ستغيّر AL.
14–16	الوظيفة 02_H تكتب الحرف الموجود في DL.
20	`13, 10`: الرجوع إلى بداية السطر ثم سطر جديد.

10.3 إدارة المكدس

تعريف

المكدس (Pile / Stack): منطقة من الذاكرة نضع فيها القيم ونأخذها بقاعدة **آخر من يدخل أول من يخرج** (LIFO: Last In, First Out). قمته محددة بـSS:SP.

تشبيه من الحياة اليومية

المكدس مثل **كومة صحون**: نضع الصحن الجديد فوق الكومة، ونأخذ دائمًا الصحن الذي في الأعلى.

- `push ax`: SP ← SP - 2، ثم نكتب AX في [SP].
- `pop ax`: نقرأ [SP] في AX، ثم SP ← SP + 2.

انتبه

في 8086 المكدس **ينمو نحو العناوين الصغيرة**: كل `push` ينقص SP. ونضع ونأخذ دائمًا **كلمات** (16 بتًا).

```
1  ; The stack: PUSH and POP
2  org 100h
3
4      mov ax, 1111h
5      mov bx, 2222h
6      push ax           ; SP = SP - 2 ; [SP] = AX
7      push bx           ; SP = SP - 2 ; [SP] = BX
8      pop ax            ; AX = [SP]   ; SP = SP + 2  -> AX = 2222h
9      pop bx            ; BX = [SP]   ; SP = SP + 2  -> BX = 1111h
10     ret               ; AX and BX have been swapped
```



السطر	التعليمة	AX	BX	SP
4–5	`mov ax/bx`	1111	2222	FFFE
6	`push ax`	1111	2222	FFFC
7	`push bx`	1111	2222	FFFA
8	`pop ax`	2222	2222	FFFC
9	`pop bx`	2222	1111	FFFE

بما أن آخر قيمة دخلت (2222_H) هي أول قيمة خرجت، فقد تبادل AX وBX قيمتيهما.

1.10.3 لماذا نحتاج إلى المكدس؟

1) **حفظ عناوين العودة**: عندما نستدعي إجراء، يحفظ المعالج في المكدس أين يجب أن يعود.
2) **حفظ السجلات مؤقتًا**: إجراء يحتاج إلى CX وهو مستعمل في حلقة؟ نكتب `push cx` في بدايته و`pop cx` في نهايته، فلا يضيع عدّاد الحلقة.
3) **تمرير المعاملات** إلى الإجراءات (سنراه بعد قليل).
4) **المقاطعات**: المعالج يحفظ فيه حالته قبل أن يذهب إلى معالج المقاطعة.

انتبه

يجب أن يكون عدد `pop` مساويًا لعدد `push`، وبالترتيب المعاكس. `push ax` ثم `push bx` يجب أن يقابلهما `pop bx` ثم `pop ax`.
إذا نسيت `pop` واحدًا داخل إجراء، فإن `ret` ستأخذ قيمة خاطئة كعنوان عودة، فيقفز البرنامج إلى مكان عشوائي.

11.3 الدوال والإجراءات

تعريف

الإجراء (Procédure, sous-programme / Procedure): مجموعة تعليمات لها اسم، نستدعيها بـ`call` ونعود منها بـ`ret`. نكتبها مرة واحدة ونستعملها عدة مرات.

1.11.3 كيف يعرف `ret` إلى أين يعود؟

- `call P`: يضع في المكدس **عنوان التعليمة التي بعده** (IP)، ثم يقفز إلى P.
- `ret`: يأخذ هذا العنوان من المكدس ويضعه في IP، فيعود التنفيذ بعد `call`.

خطوة بخطوة: الإستدعاء في البرنامج الآتي (العناوين الحقيقية)

1) التعليمة `call print_num` الأولى في الإزاحة 0103_H وطولها 3 بايتات، إذن التعليمة التي بعدها في 0106_H. يضع المعالج 0106_H في المكدس، ثم يقفز إلى `print_num`.
2) في نهاية الإجراء، `ret` تأخذ 0106_H من المكدس: يعود التنفيذ إلى `mov ax, 7`.
3) الاستدعاء الثاني في 0109_H، فيضع $010C_H$، و`ret` تعود هذه المرة إلى $010C_H$.

الإجراء نفسه يعود إلى مكانين مختلفين، لأن عنوان العودة محفوظ في المكدس في كل مرة.

2.11.3 إجراء يطبع عددًا بالعشري

المقاطعة `int 21h` تطبع حروفًا فقط. لكي نطبع العدد 2026 يجب أن نستخرج أرقامه واحدًا واحدًا:

خطوة بخطوة: استخراج أرقام 2026

العملية	الحاصل (AX)	الباقي (DX)	المكدس
2026 ÷ 10	202	6	6
202 ÷ 10	20	2	2، 6
20 ÷ 10	2	0	0، 2، 6
2 ÷ 10	0 (نتوقف)	2	2، 0، 2، 6

الأرقام تخرج بالعكس (6، 2، 0، 2). نضعها في المكدس، ثم نأخذها: فتخرج بالترتيب الصحيح 2، 0، 2، 6.

```
; A procedure that prints the number in AX in decimal
org 100h

    mov ax, 2026
    call print_num    ; print 2026
    mov ax, 7
    call print_num    ; print 7
    ret

print_num proc        ; input: AX = number to print
    mov bx, 10        ; we divide by 10
    mov cx, 0         ; CX = number of digits
divide:
    mov dx, 0         ; DX:AX / BX needs DX = 0
    div bx            ; AX = AX / 10 , DX = last digit
    push dx           ; keep the digit on the stack
    inc cx
    cmp ax, 0
    jne divide        ; continue while AX <> 0
show:
    pop dx            ; digits come back in the right order
    add dl, '0'       ; digit -> ASCII character
    mov ah, 02h
    int 21h
    loop show
    mov dl, ' '       ; a space after the number
    int 21h
    ret
print_num endp
```

screen

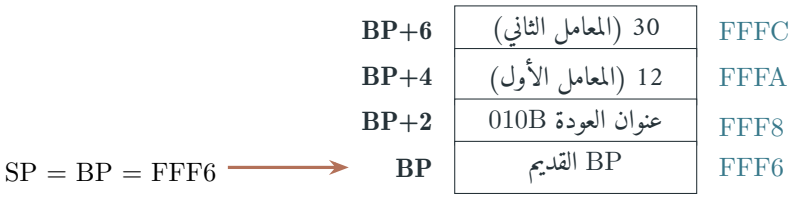
```
2026 7
```

السطر	ماذا يفعل؟
4–5	نضع العدد في AX (المعامل)، ثم نستدعي الإجراء.
10	`proc` و`endp` يحددان بداية الإجراء ونهايته.
14	DX = 0 ضروري قبل `div bx` (القسمة 16 بتًا تقسم DX:AX).
16–17	نضع الرقم في المكدس، ونعدّ الأرقام في CX.
21–25	CX مرة: نأخذ رقمًا، نحوله إلى حرف بإضافة `'0'` (48)، ونطبعه.
28	`ret` تعود إلى السطر 6 في المرة الأولى، وإلى السطر 8 في المرة الثانية.

3.11.3 تمرير المعاملات عبر المكدس

```
1  ; Passing parameters on the stack
2  org 100h
3
4      mov ax, 30
5      push ax           ; 2nd parameter
6      mov ax, 12
7      push ax           ; 1st parameter
8      call addtwo       ; result in AX
9      mov [res], ax     ; res = 42
10     ret
11
12 addtwo proc
13     push bp           ; save BP
14     mov bp, sp        ; BP = top of the stack
15     mov ax, [bp+4]    ; 1st parameter (12)
16     add ax, [bp+6]    ; + 2nd parameter (30)
17     pop bp            ; restore BP
18     ret 4             ; return and remove the 2 parameters
19 addtwo endp
20
21 res dw 0
```

بعد `mov bp, sp` في السطر 14، يبدو المكدس هكذا (القيم من التنفيذ الحقيقي):



السطر	ماذا يفعل؟
4–7	نضع المعاملين في المكدس: SP ينتقل من $_{\text{H}}$FFFE إلى $_{\text{H}}$FFFA.
8	`call` يضع عنوان العودة $_{\text{H}}$010B: SP = $_{\text{H}}$FFF8.
13–14	نحفظ BP، ثم نجعل BP ثابتًا على قمة المكدس ($_{\text{H}}$FFF6).
15–16	المعامل الأول في [BP+4]، والثاني في [BP+6]: 12 + 30 = 42.
18	`ret 4`: نعود، ثم نزيل 4 بايتات (المعاملين) من المكدس. SP يعود إلى $_{\text{H}}$FFFE.

12.3 المقاطعات

تعريف

المقاطعة (Interruption / Interrupt): حدث يوقف البرنامج الحالي مؤقتًا، لينفذ المعالج برنامجًا خاصًا (**معالج المقاطعة**)، ثم يعود ليكمل من حيث توقف.

تشبيه من الحياة اليومية

أنت تقرأ كتابًا، فيرن **جرس الباب**. تضع علامة في الصفحة، تفتح الباب، ثم تعود إلى القراءة من الصفحة نفسها. الجرس هو المقاطعة، والعلامة في الكتاب هي ما يحفظه المعالج في المكدس.

1.12.3 أنواع المقاطعات

النوع	من يسببها؟	أمثلة
عتادية	جهاز خارجي يرسل إشارة على المنفذ INTR أو NMI	ضغط مفتاح، الساعة، الطابعة جاهزة
برمجية	التعليمة `int n` في البرنامج	`int 21h` لخدمات DOS
استثناءات	المعالج نفسه عند خطأ	القسمة على صفر (مقاطعة 0)

2.12.3 جدول أشعة المقاطعات

في بداية الذاكرة (من 00000_H إلى $003FF_H$) يوجد جدول من 256 خانة، لكل مقاطعة 4 بايتات: الإزاحة ثم المقطع لمعالج المقاطعة.

تعريف

شعاع المقاطعة رقم n يوجد في العنوان: $4 \times n$

مثال

شعاع `int 21h`: $21_H \times 4 = 33 \times 4 = 132 = 84_H$. إذن عنوان معالج DOS محفوظ في 00084_H إلى 00087_H.

3.12.3 ماذا يفعل المعالج عند `int n`؟

1) يضع FLAGS في المكدس، ثم يطفئ IF وTF.
2) يضع CS ثم IP في المكدس (العلامة في الكتاب).
3) يقرأ IP وCS الجديدين من الخانة $4 \times n$، فيقفز إلى معالج المقاطعة.
4) في نهاية المعالج، `iret` تأخذ IP وCS وFLAGS من المكدس، فيعود البرنامج.

4.12.3 المقاطعة العتادية خطوة بخطوة

خطوة بخطوة: تضغط على مفتاح أثناء تشغيل برنامج

1) لوحة المفاتيح ترسل إشارة إلى **متحكم المقاطعات** (Contrôleur d'interruptions / PIC 8259).
2) المتحكم يرسل إشارة على المدخل INTR للمعالج.
3) إذا كان IF = 1، يُكمل المعالج **التعليمة الحالية** أولًا، ثم يقبل المقاطعة.
4) المتحكم يعطي المعالج رقم المقاطعة (09_H للوحة المفاتيح).
5) المعالج يحفظ FLAGS وCS وIP، ثم يقفز إلى العنوان المكتوب في $4 \times 9 = 24_H$.
6) معالج المقاطعة يقرأ المفتاح من المنفذ 60_H، يحفظه، ثم ينفذ `iret`.
7) البرنامج يكمل من حيث توقف، دون أن «يشعر» بشيء.

تعريف

- `cli` (clear IF): تمنع المقاطعات العتادية مؤقتًا، و`sti` (set IF) تعيد السماح بها. نستعملهما عندما يجب ألّا يُقاطَع جزء حساس من البرنامج.
- المدخل NMI (Non Maskable Interrupt): مقاطعة **لا يمكن منعها**، تُستعمل للأخطاء الخطيرة مثل عطل في الذاكرة أو انقطاع التيار.

5.12.3 برنامج: مقاطعتنا الخاصة

```
; Install our own interrupt handler for INT 60h
org 100h

    mov ax, 0
    mov es, ax            ; ES = 0 : the interrupt vector table
    lea ax, handler
    mov es:[60h*4], ax    ; offset of the handler
    mov es:[60h*4+2], cs  ; segment of the handler
    mov bl, 'A'
    int 60h               ; call our interrupt
    int 60h               ; call it again
    ret

handler:                  ; prints BL, then BL = next letter
    mov dl, bl
    mov ah, 02h
    int 21h
    inc bl
    iret                  ; return from interrupt (restores IP, CS, FLAGS)
```

screen
```
AB
```

السطر	ماذا يفعل؟
4–5	ES = 0: نريد الكتابة في جدول الأشعة الموجود في المقطع 0.
6–8	نكتب عنوان `handler` في الخانة $60_H \times 4 = 180_H$: الإزاحة ثم المقطع.
10–11	كل `int 60h` ينفذ `handler`: المرة الأولى يطبع `A`، والثانية `B`.
19	`iret` وليس `ret`، لأن المقاطعة وضعت في المكدس FLAGS أيضًا.

13.3 تمارين

تمرين 1.3

احسب العنوان الفيزيائي في كل حالة: (أ) DS = 3000H، BX = 0050H مع `mov al, [bx]`. (ب) SS = 4000H، SP = FFFEH.

الحل

(أ) $30000_H + 0050_H = 30050_H$. (ب) $40000_H + FFFE_H = 4FFFE_H$.

تمرين 2.3

ما قيمة AX بعد هذه التعليمات؟ `mov ax, 0F0Fh` ثم `and ax, 00FFh` ثم `shl ax, 1`.

الحل

بعد `and`: $000F_H$. بعد الإزاحة (ضرب في 2): $001E_H = 30$.

تمرين 3.3

اكتب برنامجًا يحسب $5! = 1 \times 2 \times 3 \times 4 \times 5$ بالتعليمة `loop`، ويضع النتيجة في AX.

الحل

```
; Exercise: 5! with LOOP
org 100h

    mov ax, 1          ; result = 1
    mov cx, 5          ; multiply by 5, 4, 3, 2, 1
again:
    mul cx             ; DX:AX = AX * CX
    loop again
    ret                ; AX = 120
```

النتيجة: 0078 = AX = 120 (جربناه في المحاكي).

تمرين 4.3

بعد `mov sp, 0100h` ثم `push ax` ثم `push bx` ثم `pop cx`: ما قيمة SP، وأي قيمة في CX؟

الحل

بعد الدفعتين $SP = 00FC_H$، وبعد `pop` $SP = 00FE_H$. CX يأخذ آخر قيمة دخلت: قيمة BX.

تمرين 5.3

ما عنوان شعاع المقاطعة رقم 10_H (خدمات الشاشة في BIOS)؟

الحل

$10_H \times 4 = 16 \times 4 = 64 = 40_H$: الخانات من 00040_H إلى 00043_H.

تمرين 6.3

فك ترميز التعليمة 89 D8.

الحل

89: `mov` 16 بتًا، $d = 0$. D8 = $11\,011\,000_2$: $mod = 11$ (سجل)، $reg = 011$ (BX، المصدر)، $r/m = 000$ (AX).
إذن: `mov ax, bx`.

الفصل 4

دراسة حالة: المعالجات ذات 32 و64 بتًا

خلاصة: ماذا سنتعلم في هذا الفصل؟

- لماذا انتقلنا من 16 بتًا إلى 32 ثم 64 بتًا، وما الذي تغيّر.
- تطور المعمارية من IA-32 إلى AMD64.
- سجلات المعالجات الحديثة، والعنونة المسطحة.
- تعليمات جديدة: SSE وAVX التي تحسب عدة أعداد في وقت واحد.
- خط الأنابيب: كيف ينفذ المعالج عدة تعليمات في الوقت نفسه.

انتبه

برامج هذا الفصل مكتوبة بصيغة **NASM** لنظام **Linux**، وقد نفذناها فعلًا على معالج Intel Core i7-8665U (4 أنوية). للتجميع والتنفيذ: `nasm -f elf64 prog.asm` ثم `ld prog.o -o prog` ثم `./prog`.

1.4 من 16 إلى 32 ثم 64 بتًا

تعريف

نقول إن المعالج من n بتًا عندما تكون **سجلاته العامة** من n بتًا: هذا هو حجم العدد الذي يحسبه في تعليمة واحدة، وحجم العنوان الذي يستطيع وضعه في سجل.

1.1.4 الفروق الأساسية

	16 بتًا (8086)	32 بتًا (IA-32)	64 بتًا (x86-64)
أكبر عدد دون إشارة	$2^{16}-1 = 65\,535$	$2^{32}-1 \approx 4.3$ مليار	$2^{64}-1 \approx 1.8 \times 10^{19}$
الذاكرة القصوى	1 MB (بالمقاطع)	4 GB	256 TB (48 بتًا عمليًا)
عدد السجلات العامة	8	8	**16**
أسماء السجلات	AX, BX, …	EAX, EBX, …	RAX, RBX, …, R15
نمط العنونة	مقطعي	مسطح (عمليًا)	مسطح

2.1.4 لماذا نحتاج إلى أكثر من 32 بتًا؟

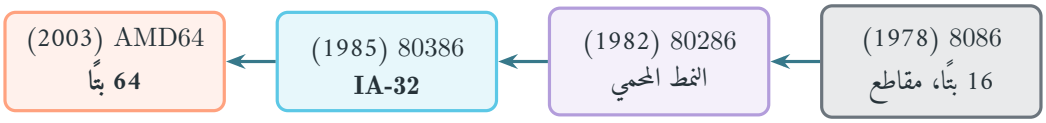
1) **الذاكرة**: بعنوان من 32 بتًا لا نستطيع استعمال أكثر من 4 GB. حواسيب اليوم فيها 16 أو 32 GB، والخوادم فيها مئات الغيغابايتات.
2) **الأداء**: سجل من 64 بتًا يحسب عددًا كبيرًا في تعليمة واحدة، بينما يحتاج معالج 32 بتًا إلى تعليمتين على الأقل (كما رأينا `add` و`adc` في الفصل الثالث). ومضاعفة عدد السجلات (16 بدل 8) تقلل الذهاب إلى الذاكرة.
3) **التوافق**: المعالج من 64 بتًا ينفذ أيضًا البرامج القديمة ذات 32 بتًا دون تغيير.

خطوة بخطوة: ما معنى «4 GB» بالضبط؟

1) عنوان من 32 بتًا يأخذ 2^{32} قيمة مختلفة.
2) $2^{32} = 2^2 \times 2^{30} = 4 \times 2^{30}$.
3) و2^{30} بايت $= 1$ GB، إذن السعة القصوى $= 4$ GB.

بـ48 بتًا: $2^{48} = 2^8 \times 2^{40} = 256$ TB، أي أكبر بـ65536 مرة!

2.4 تطور المعمارية: من IA-32 إلى AMD64



- **IA-32** (Intel Architecture 32 bits): بدأت مع 80386. السجلات أصبحت من 32 بتًا (EAX، الحرف E يعني Extended)، وظهر **النمط المحمي** الذي يمنع البرامج من الكتابة في ذاكرة بعضها البعض.
- **AMD64**: المفاجأة أن شركة AMD (وليس Intel) هي التي صممت الامتداد إلى 64 بتًا سنة 2003، مع المحافظة على التوافق. تبنّته Intel بعد ذلك باسم Intel 64. نسميه اليوم x86-64.

1.2.4 أنماط عمل المعالج

النمط	الوصف	متى؟
الحقيقي (real mode)	يتصرف كمعالج 8086: 16 بتًا و1 MB	عند تشغيل الحاسوب، قبل نظام التشغيل
المحمي (protected mode)	32 بتًا، حماية الذاكرة	أنظمة 32 بتًا
الطويل (long mode)	64 بتًا، ومعه نمط فرعي للتوافق مع برامج 32 بتًا	كل الأنظمة الحديثة

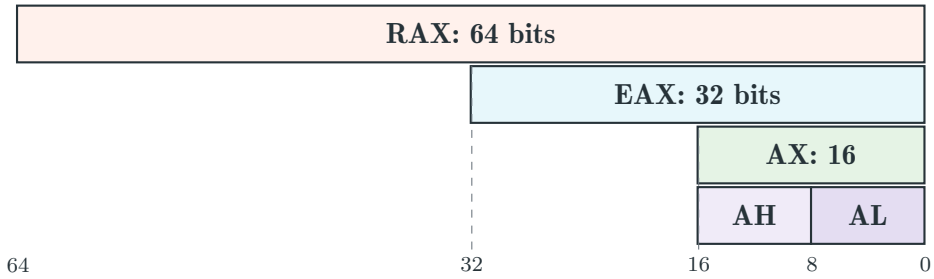
هل تعلم؟

كل حاسوب حديث يستيقظ وهو «يعتقد» أنه 8086! يبدأ في النمط الحقيقي، ثم ينقله نظام التشغيل إلى النمط المحمي ثم الطويل. هكذا تبقى المعمارية متوافقة منذ 1978.

3.4 السجلات في معالجات 32 و64 بتًا

1.3.4 السجلات المتداخلة

كل سجل جديد يحتوي السجل القديم في نصفه الأيمن (البتات الصغيرة):



64 بتًا	32 بتًا	16 بتًا	8 بتات	الاستعمال المعتاد
RAX	EAX	AX	AL	النتيجة المعادة من دالة
RBX	EBX	BX	BL	عام
RCX	ECX	CX	CL	العدّاد، المعامل الرابع
RDX	EDX	DX	DL	المعامل الثالث
RSI	ESI	SI	SIL	المعامل الثاني
RDI	EDI	DI	DIL	المعامل الأول
RSP	ESP	SP	SPL	قمة المكدس
RBP	EBP	BP	BPL	قاعدة إطار المكدس
R8 ...R15	R8D ...R15D	R8W ...	R8B ...	**جديدة في 64 بتًا**

- **RIP**: مؤشر التعليمة (كان IP ثم EIP).
- **RFLAGS**: سجل الأعلام، والأعلام ZF، CF، SF، OF في المكان نفسه كما في 8086.
- **سجلات المقاطع**: ما زالت موجودة (CS، DS...) لكن بدايتها كلها 0 في 64 بتًا، ما عدا FS وGS التي يستعملها النظام لمعطيات خاصة بكل خيط تنفيذ.

2.3.4 تجربة: ماذا يحدث للبتات الأخرى؟

```
; RAX, EAX, AX, AL: what happens to the other bits?
section .text
global _start
_start:
    mov rax, 0x1122334455667788
    call print_hex          ; the full 64-bit value

    mov rax, 0x1122334455667788
    mov eax, 0xAABBCCDD     ; write 32 bits: the high half becomes 0!
    call print_hex

    mov rax, 0x1122334455667788
    mov ax, 0x1234          ; write 16 bits: the rest is kept
    call print_hex

    mov rax, 0x1122334455667788
    mov al, 0xFF            ; write 8 bits: the rest is kept
    call print_hex

    mov rax, 60             ; exit
    xor rdi, rdi
    syscall

print_hex:                  ; print RAX as 16 hex digits
    mov rcx, 16             ; 16 digits of 4 bits
    lea rdi, [rel buf]
next_digit:
    rol rax, 4              ; bring the next 4 bits to the right
    mov rdx, rax
    and rdx, 0xF            ; keep only these 4 bits
    mov dl, [digits + rdx]  ; 0..15 -> '0'..'F'
    mov [rdi], dl
    inc rdi
    dec rcx
    jnz next_digit
    mov byte [rdi], 10      ; new line
    mov rax, 1              ; write(1, buf, 17)
    mov rdi, 1
    lea rsi, [rel buf]
    mov rdx, 17
    syscall
    ret

section .data
digits  db "0123456789ABCDEF"

section .bss
buf     resb 17
```

```
screen
1122334455667788
00000000AABBCCDD
1122334455661234
11223344556677FF
```

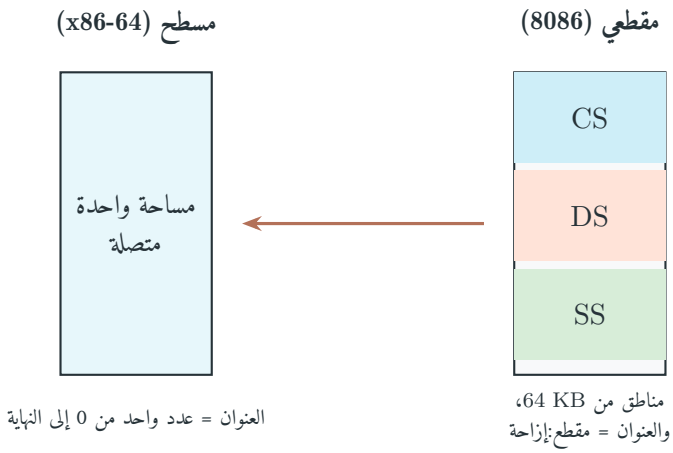
السطر	ماذا يفعل؟
5–6	نضع عددًا من 64 بتًا في RAX ونعرضه: 1122334455667788.
9	الكتابة في EAX (32 بتًا) **تصفّر** النصف الأعلى: 00000000AABBCCDD.
13	الكتابة في AX (16 بتًا) تترك باقي البتات كما هي: 5566...**1234**.
17	الكتابة في AL تغيّر البايت الأخير فقط: 77...**FF**.
24–42	إجراء يعرض RAX بالست عشري: في كل دورة يدوّر 4 بتات، يأخذها بالقناع `0xF`، ويحوّلها إلى حرف.

انتبه

هذه قاعدة غريبة لكنها مهمة: في 64 بتًا، **كل كتابة في سجل من 32 بتًا تصفّر البتات العليا**، أما الكتابة في 8 أو 16 بتًا فلا تفعل ذلك. صممت AMD هذه القاعدة لتسريع المعالج.

4.4 أنماط العنونة المتقدمة

1.4.4 من العنونة المقطعية إلى العنونة المسطحة



في **النموذج المسطح** (Modèle mémoire plat / Flat memory model) يرى البرنامج الذاكرة **كشريط واحد طويل** من البايتات، مرقمة من 0. لا نحتاج إلى حساب «مقطع × 16 + إزاحة»: العنوان هو مجرد عدد في سجل من 64 بتًا.

تشبيه من الحياة اليومية

العنونة المقطعية تشبه أن تقول: «العمارة 12، الطابق 3». العنونة المسطحة تشبه أن تعطي لكل شقة في المدينة رقمًا واحدًا فريدًا: «الشقة رقم 73 502». أبسط بكثير، بشرط أن نملك أعدادًا كبيرة بما يكفي.

2.4.4 الشكل العام للعنوان

تعريف

العنوان = القاعدة + الفهرس × السلّم + الإزاحة

القاعدة والفهرس: أي سجلين. السلّم: 1 أو 2 أو 4 أو 8. الإزاحة: عدد ثابت.

الكتابة (NASM)	الاستعمال
`[rbx]`	القاعدة فقط: مؤشر
`[rbx + 16]`	قاعدة + إزاحة: حقل في هيكل (struct)
`[tab + rcx*8]`	فهرس × سلّم + إزاحة: العنصر i من جدول أعداد من 8 بايتات
`[rbx + rcx*4 + 8]`	كل العناصر معًا
`[rel msg]`	نسبي إلى RIP: خاص بـ64 بتًا

خطوة بخطوة: أين يقرأ `[tab + rcx*8]` عندما RCX = 3؟

1) كل عنصر من `tab` عدد من 64 بتًا = 8 بايتات (`dq`).
2) العنصر رقم 3 (الرابع، لأننا نبدأ من 0) على بعد $3 \times 8 = 24$ بايتًا من البداية.
3) إذا كان `tab` في العنوان 402000_{H}، فالعنوان هو $402000_{\text{H}} + 24 = 402018_{\text{H}}$.

السلّم يُغنينا عن ضرب الفهرس بأنفسنا: المعالج يفعل ذلك داخل التعليمة نفسها.

```asm
; Sum of an array of 64-bit numbers: base + index*scale addressing
section .text
global _start
_start:
    xor rax, rax                ; sum = 0
    xor rcx, rcx                ; i = 0
again:
    add rax, [tab + rcx*8]      ; sum += tab[i]   (8 bytes per element)
    inc rcx                     ; i++
    cmp rcx, 5
    jl  again                   ; while i < 5
    call print_dec              ; print the sum

    mov rax, 60
    xor rdi, rdi
    syscall

print_dec:                      ; print RAX in decimal
    lea rsi, [rel dbuf + 20]    ; fill the buffer from the end
    mov byte [rsi], 10          ; new line
    mov rbx, 10
    mov rcx, 1                  ; length (the new line)
divide:
    xor rdx, rdx
    div rbx                     ; RAX = RAX / 10, RDX = digit
    add dl, 48                  ; digit -> ASCII ('0' = 48)
    dec rsi
    mov [rsi], dl
    inc rcx
    test rax, rax
    jnz divide
    mov rax, 1                  ; write(1, rsi, rcx)
    mov rdi, 1
    mov rdx, rcx
    syscall
    ret

section .data
tab     dq 1000000000000, 2500, 30000000000, 7, 12345

section .bss
dbuf    resb 21
```

screen

```
1030000014852
```

السطر	ماذا يفعل؟
5–6	`xor` سجل مع نفسه = 0: المجموع والفهرس يبدآن من 0.
8	العنونة بالسلّم: نقرأ العنصر i من الجدول، ونجمعه في RAX.
9–11	`i++` ثم «طالما $i < 5$» نعود. هذه حلقة `while` كما رأينا في الفصل الثالث.
18–36	نفس طريقة الفصل الثالث لعرض عدد بالعشري (القسمة على 10)، لكن بسجلات 64 بتًا.
39	`dq` (define quad-word): أعداد من 64 بتًا. العدد الأول (10^{12}) لا يدخل في 32 بتًا.

3.4.4 العنونة النسبية إلى RIP

في 64 بتًا نكتب كثيرًا `lea rsi, [rel buf]`: العنوان يُحسب **بالنسبة إلى التعليمة الحالية** (مسافة + RIP). الفائدة: يعمل البرنامج في أي مكان يضعه فيه النظام في الذاكرة، وهذا مهم للأمان (النظام يغيّر مكان البرامج في كل تشغيل لكي يصعب على المهاجمين معرفة العناوين).

5.4 مجموعة التعليمات

1.5.4 التعليمة نفسها على 16 و32 و64 بتًا

جمّعنا البرنامج الآتي، وهذه بايتات لغة الآلة الحقيقية:

```
1  ; The same addition on 16, 32 and 64 bits
2  section .text
3  global _start
4  _start:
5      add ax, bx          ; 16 bits
6      add eax, ebx        ; 32 bits
7      add rax, rbx        ; 64 bits
8      add r8, r9          ; new 64-bit registers
9      jmp fin             ; a short jump
10 fin:
11     mov rax, 60
12     xor rdi, rdi
13     syscall
```

التعليمة	لغة الآلة	الملاحظة
`add ax, bx`	`66 01 D8`	البادئة 66: «انتبه، 16 بتًا»
`add eax, ebx`	`01 D8`	32 بتًا هو الحجم الافتراضي
`add rax, rbx`	`48 01 D8`	البادئة REX.W = 48: «64 بتًا»
`add r8, r9`	`4D 01 C8`	REX = 4D: 64 بتًا + سجلات جديدة

خطوة بخطوة: لماذا 01 D8 موجودة في الأسطر الثلاثة الأولى؟

1) 01 هو الرمز العملي «جمع سجل في سجل أو ذاكرة»، وهو نفسه منذ 8086.
2) D8 = $11\,011\,000_2$ هو بايت ModR/M الذي رأيناه في الفصل الثالث: mod = 11 (سجل)، reg = 011 (BX)، r/m = 000 (AX).
3) الحجم (16 أو 32 أو 64) لا يُكتب في التعليمة نفسها، بل في بادئة قبلها. هكذا بقيت التعليمات القديمة صالحة، وأضيفت الأحجام الجديدة «من الخارج».
4) في 4D = $0100\,1101_2$: 0100 تعني «بادئة REX»، ثم بت W = 1 (64 بتًا)، وبتّان يضيفان 8 إلى رقم السجلين، فيصبح AX هو R8 وBX هو R9.

2.5.4 مقارنة: برنامج 32 بتًا وبرنامج 64 بتًا

بـ32 بتًا:

```
; The same program in 32-bit Linux (NASM syntax)
section .data
msg     db "Hello from 32-bit mode!", 10
len     equ $ - msg

section .text
global _start
_start:
    mov eax, 4          ; system call 4 = write (32-bit numbers differ!)
    mov ebx, 1          ; stdout
    mov ecx, msg        ; address of the text
    mov edx, len        ; number of bytes
    int 0x80            ; 32-bit way to call Linux

    mov eax, 1          ; system call 1 = exit
    mov ebx, 0
    int 0x80
```

screen

```
Hello from 32-bit mode!
```

بـ64 بتًا:

```asm
; Hello in 64-bit Linux (NASM syntax)
section .data
msg     db "Hello from 64-bit mode!", 10    ; text + new line
len     equ $ - msg                          ; length computed by NASM

section .text
global _start
_start:
    mov rax, 1          ; system call 1 = write
    mov rdi, 1          ; file 1 = the screen (stdout)
    mov rsi, msg        ; address of the text
    mov rdx, len        ; number of bytes
    syscall             ; ask Linux to do it

    mov rax, 60         ; system call 60 = exit
    mov rdi, 0          ; exit code 0 = success
    syscall
```

screen

```
Hello from 64-bit mode!
```

	32 بتًا	64 بتًا
طلب خدمة من النظام	`int 0x80` (مقاطعة)	`syscall` (تعليمة خاصة، أسرع)
رقم الخدمة write	4 في EAX	1 في RAX
المعاملات	EBX، ECX، EDX	RDI، RSI، RDX
حجم العنوان	4 بايتات	8 بايتات

هل تعلم؟

نفذنا البرنامجين على الحاسوب نفسه ذي 64 بتًا. البرنامج ذو 32 بتًا عمل دون أي تعديل: هذا هو النمط الفرعي للتوافق داخل النمط الطويل.

3.5.4 ماذا يكتب المترجم؟ من C إلى لغة التجميع

كتبنا دالة بلغة C، وطلبنا من المترجم gcc أن يعطينا لغة التجميع التي يولّدها (`gcc -O1 -S -masm=intel sum.c`):

```asm
sum:
    test rsi, rsi
    jle .L4
    mov rax, rdi
    lea rcx, [rdi+rsi*8]
    mov edx, 0
.L3:
    add rdx, QWORD PTR [rax]
    add rax, 8
    cmp rax, rcx
    jne .L3
.L1:
    mov rax, rdx
    ret
.L4:
    mov edx, 0
    jmp .L1
```

```c
/* compiled by gcc for x86-64 */
long sum(long *t, long n)
{
    long s = 0;
    for (long i = 0; i < n; i++)
        s += t[i];
    return s;
}
```

السطر	ماذا يفعل؟
1	حسب قواعد Linux لـ64 بتًا: المعامل الأول (t) يصل في RDI، والثاني (n) في RSI.
2–3	إذا كان $n \leq 0$ نقفز إلى L4.: النتيجة 0.
5	lea: يحسب عنوان نهاية الجدول $t + n \times 8$ **دون أن يقرأ الذاكرة.**
8–11	الحلقة: نجمع العنصر، نتقدم بـ8 بايتات، ونتوقف عند النهاية. المترجم استعمل مؤشرًا بدل الفهرس i.
13–14	النتيجة تُعاد دائمًا في RAX.

4.5.4 SSE وAVX: حساب عدة أعداد مرة واحدة

تعريف

SIMD (Une instruction, données multiples / Single Instruction, Multiple Data): تعليمة واحدة تطبق العملية نفسها على **عدة أعداد في الوقت نفسه.**

4.5	3.5	2.5	1.5	XMM0 (128 بتًا = 4 أعداد)
40	30	20	10	+
44.5	33.5	22.5	11.5	النتيجة: **تعليمة واحدة** addps

الامتداد	السنة	السجلات	الحجم	أعداد float في المرة
SSE	1999	XMM0–XMM15	128 بتًا	4
AVX	2011	YMM0–YMM15	256 بتًا	8
AVX-512	2016	ZMM0–ZMM31	512 بتًا	16

```c
/* SSE: add 4 floats with one instruction */
#include <stdio.h>
#include <xmmintrin.h>

int main(void)
{
    float a[4] = {1.5, 2.5, 3.5, 4.5};
    float b[4] = {10, 20, 30, 40};
    float c[4];

    __m128 x = _mm_loadu_ps(a);   /* load 4 floats into XMM */
    __m128 y = _mm_loadu_ps(b);
    __m128 z = _mm_add_ps(x, y);  /* ONE instruction: addps */
    _mm_storeu_ps(c, z);

    printf("%.1f %.1f %.1f %.1f\n", c[0], c[1], c[2], c[3]);
    return 0;
}
```

screen

```
11.5 22.5 33.5 44.5
```

عندما نفتح البرنامج المترجم، نجد فعلًا التعليمة الحقيقية:

```
addps  xmm0, XMMWORD PTR [rbp-0x50]   ; 4 additions at once
```

ونسخة AVX تجمع 8 أعداد بالتعليمة `vaddps ymm0, ymm0, […]`:

```c
/* AVX: add 8 floats with one instruction */
#include <stdio.h>
#include <immintrin.h>

int main(void)
{
    float a[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    float b[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    float c[8];

    __m256 x = _mm256_loadu_ps(a);   /* 8 floats in a YMM register */
    __m256 y = _mm256_loadu_ps(b);
    _mm256_storeu_ps(c, _mm256_add_ps(x, y));   /* vaddps */

    for (int i = 0; i < 8; i++)
        printf("%.0f ", c[i]);
    printf("\n");
    return 0;
}
```

screen

```
11 22 33 44 55 66 77 88
```

تشبيه من الحياة اليومية

التعليمة العادية تشبه أستاذًا يصحح ورقة امتحان واحدة في كل مرة. SIMD تشبه أستاذًا يضع 8 أوراق جنبًا إلى جنب، ويصحح السؤال الأول فيها كلها بحركة واحدة. لهذا تُستعمل في معالجة الصور، الفيديو، الألعاب، والذكاء الاصطناعي، حيث نطبق العملية نفسها على ملايين الأعداد.

6.4 دورة تنفيذ التعليمة وخط الأنابيب

1.6.4 المراحل الخمس

في المعالجات الحديثة، تنقسم كل تعليمة إلى مراحل، ولكل مرحلة دارة خاصة بها:

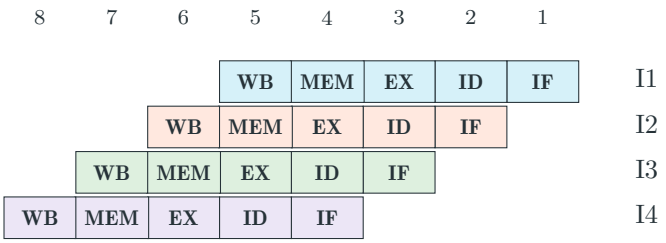
المرحلة	الاسم	العمل
IF	الجلب	قراءة التعليمة من الذاكرة (بعنوان RIP)
ID	فك الترميز	فهم التعليمة وقراءة السجلات
EX	التنفيذ	حساب النتيجة في UAL أو حساب العنوان
MEM	الذاكرة	القراءة أو الكتابة في الذاكرة إن احتجنا
WB	الكتابة	حفظ النتيجة في السجل

2.6.4 دون خط أنابيب ومع خط أنابيب

تشبيه من الحياة اليومية

في مغسلة ملابس: الغسل ساعة، التجفيف ساعة، الكي ساعة. لغسل 3 أكياس واحدًا بعد الآخر نحتاج 9 ساعات. لكن عندما يخرج الكيس الأول من الغسالة إلى المجفف، نضع الكيس الثاني في الغسالة فورًا. الآلات الثلاث تعمل معًا، فننتهي في 5 ساعات فقط. هذا هو خط الأنابيب.

مع خط الأنابيب: 4 تعليمات في 8 دورات



تعريف

لـ n تعليمة وk مرحلة (كل مرحلة دورة واحدة):

- دون خط أنابيب: $n \times k$ دورة.

> مع خط أنابيب: $k + (n-1)$ دورة: الأولى تحتاج k دورات، ثم تنتهي تعليمة في **كل** دورة.

خطوة بخطوة: 100 تعليمة في خط أنابيب من 5 مراحل

1) دون خط أنابيب: $100 \times 5 = 500$ دورة.
2) مع خط أنابيب: $5 + (100 - 1) = 104$ دورات.
3) التسريع: $\frac{500}{104} \approx 4.8$ مرات، أي قريب من عدد المراحل (5).

3.6.4 المخاطر: متى يتوقف خط الأنابيب؟

1) **خطر المعطيات**: `add rax, rbx` ثم `add rcx, rax`. التعليمة الثانية تحتاج RAX الجديد، لكن الأولى لم تكتبه بعد. الحل: المعالج «يمرر» النتيجة مباشرة من EX إلى التعليمة الثانية، أو ينتظر دورة.
2) **خطر التحكم**: بعد `jmp` أو `jne`، التعليمات التي جُلبت بعدها قد تكون خاطئة. الحل: **التنبؤ بالقفز** (Prédiction de branchement / Branch prediction): المعالج يخمّن هل سيقفز، وإن أخطأ يرمي التعليمات الخاطئة ويبدأ من جديد.
3) **خطر البنية**: تعليمتان تحتاجان الدارة نفسها في الدورة نفسها (مثل الذاكرة).

4.6.4 مثال: `mov` و`add` و`jmp` في 64 بتًا

نتابع ثلاث تعليمات من البرنامج `same.asm` وبرامج هذا الفصل، ببايتاتها الحقيقية:

المرحلة	`mov rax, 60`	`add rax, rbx`	`jmp fin`
IF	جلب 7 بايتات 48 C7 C0 3C 00 00 00	جلب 3 بايتات 48 01 D8	جلب بايتين EB 00
ID	REX.W: 64 بتًا، الوجهة RAX، والقيمة 60 داخل التعليمة	64 بتًا، المصدر RBX، الوجهة RAX، وقراءة السجلين	قفز قصير، الإزاحة 0
EX	لا حساب (القيمة جاهزة)	RAX + RBX :UAL وحساب الأعلام	حساب الهدف: RIP + 0
MEM	لا شيء	لا شيء	لا شيء
WB	RAX ← 60	RAX يأخذ النتيجة، وتحديث RFLAGS	RIP يأخذ الهدف، ورمي ما جُلب خطأً

هل تعلم؟

معالج Core i7 الذي جربنا عليه لا يكتفي بخط أنابيب من 5 مراحل: فيه أكثر من 14 مرحلة، ويستطيع إطلاق **4 تعليمات في كل دورة**، بل وينفذها **بغير ترتيبها** إذا كانت مستقلة عن بعضها، ثم يرتب النتائج. كل هذا يحدث دون أن يرى المبرمج أي فرق.

7.4 تمارين

تمرين 1.4

كم بايتًا يستطيع معالج أن يعنون بعنوان من 36 بتًا؟ (هذا ما سمح به الامتداد PAE في معالجات 32 بتًا).

الحل

$2^{36} = 2^6 \times 2^{30} = 64$ GB.

تمرين 2.4

$\text{RAX} = \text{FFFFFFFFFFFFFFFF}_\text{H}$. ما قيمة RAX بعد: (أ) `mov eax, 1` (ب) `mov ax, 1` (ج) `mov al, 1`؟

الحل

(أ) $0000000000000001_\text{H}$ (الكتابة في 32 بتًا تصفّر النصف الأعلى).
(ب) $\text{FFFFFFFFFFFF0001}_\text{H}$. (ج) $\text{FFFFFFFFFFFFFF01}_\text{H}$.

تمرين 3.4

جدول من أعداد int (4 بايتات) يبدأ في 601000_H. ما عنوان العنصر رقم 10؟ واكتب العنونة بصيغة NASM.

الحل

$601000_\text{H} + 10 \times 4 = 601000_\text{H} + 40 = 601028_\text{H}$. الصيغة: `[tab + rcx*4]` مع $\text{RCX} = 10$.

تمرين 4.4

خط أنابيب من 6 مراحل، و1000 تعليمة. احسب عدد الدورات دون خط أنابيب ومعه، والتسريع.

الحل

دون: $6 \times 1000 = 6000$. مع: $6 + 999 = 1005$. التسريع $\frac{6000}{1005} \approx 5.97$.

تمرين 5.4

في البرنامج الآتي، أين يوجد خطر المعطيات؟ `mov rax, [x]` ثم `add rax, 5` ثم `mov [y], rax`.

الحل

الثانية تحتاج RAX الذي تقرؤه الأولى من الذاكرة، والثالثة تحتاج RAX الذي تحسبه الثانية: كل تعليمة تعتمد على التي قبلها مباشرة.

الفصل 5

بنية المعالجات الحديثة

خلاصة: ماذا سنتعلم في هذا الفصل؟

- كيف تطورت المعالجات، ولماذا توقف ارتفاع التردد.
- المعالجات متعددة الأنوية والمعالجة المتوازية: الفوائد والصعوبات، وقانون Amdahl.
- الذاكرة المخبئية وتسلسل الذاكرة، ولماذا يكون ترتيب قراءة المعطيات مهمًا جدًا.

انتبه

كل الأرقام في هذا الفصل **قسناها فعلًا** على حاسوب محمول بمعالج Intel Core i7-8665U (4 أنوية، 8 خيوط، ذاكرة مخبئية L1 = 32 KB وL2 = 256 KB لكل نواة، وL3 = 8 MB مشتركة). إذا أعدت التجارب على حاسوبك ستحصل على أرقام مختلفة قليلًا، لكن الظواهر نفسها.

1.5 المعالجات الحديثة: الإتجاهات والتطور

1.1.5 خمسون سنة من التقدم

المعالج	السنة	الترانزستورات	التردد	الأنوية
Intel 4004	1971	2 300	0.74 MHz	1
Intel 8086	1978	29 000	5–10 MHz	1
Intel 80386	1985	275 000	16–33 MHz	1
Pentium 4	2000	42 مليون	1.3–3.8 GHz	1
Core i7-8665U	2019	حوالي ملياري	1.9–4.8 GHz	4
معالجات 2024	2024	عشرات المليارات	حوالي 5 GHz	من 8 إلى أكثر من 100

2.1.5 جدار التردد

حتى سنة 2004 تقريبًا، كانت كل سنة تأتي بمعالج أسرع لأن **التردد** يرتفع. ثم توقف التردد عند حوالي 4 إلى 5 GHz. السبب الرئيسي هو الحرارة:

- كل نبضة ساعة تستهلك طاقة. كلما ارتفع التردد ارتفعت الطاقة المستهلكة، والطاقة تتحول إلى حرارة.
- عند 4 GHz تقريبًا يسخن المعالج إلى درجة لا نستطيع تبريدها في حاسوب عادي.

تشبيه من الحياة اليومية

عدّاء يركض أسرع وأسرع، حتى يصل إلى سرعة لا يستطيع أن يحافظ عليها دون أن ينهار. بدل أن نطلب منه الركض أسرع، نضيف عدّائين آخرين يتقاسمون معه المسافة. هذا بالضبط ما فعله صانعو المعالجات.

3.1.5 الإتجاهات الحالية

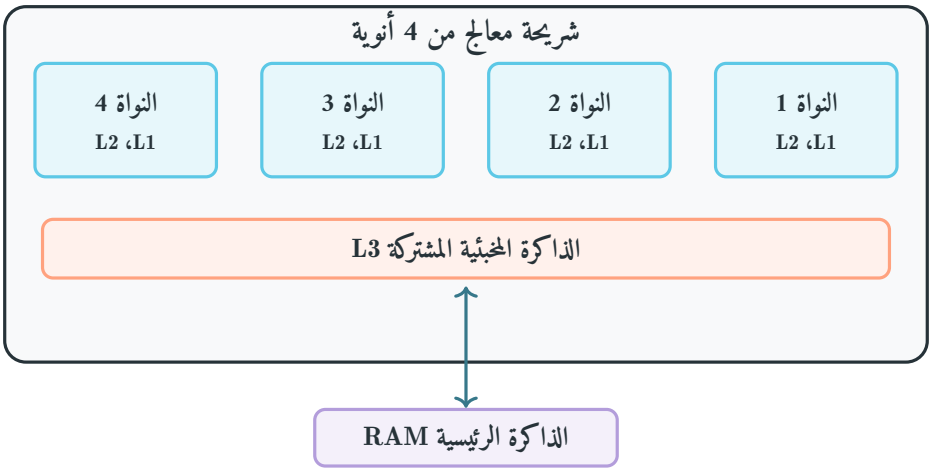
1) **أنوية أكثر** بدل تردد أعلى (القسم الآتي).
2) **ذاكرة مخبئية أكبر وأذكى**، لأن الذاكرة أصبحت أبطأ بكثير من المعالج (القسم الأخير).
3) **تعليمات متخصصة**: SIMD (الفصل الرابع)، وتسريع الذكاء الاصطناعي والتشفير.
4) **أنوية مختلفة في الشريحة نفسها**: أنوية قوية للعمل الثقيل وأنوية اقتصادية للعمل الخفيف، لتوفير البطارية.

2.5 المعالجات متعددة الأنوية والتوازي

1.2.5 المفاهيم

تعريف

- **النواة** (Cœur / Core): معالج كامل (وحدة تحكم، وحدة حساب، سجلات، ذاكرة مخبئية L1).
- **المعالج متعدد الأنوية** (Processeur multi-cœur / Multi-core processor): عدة أنوية في شريحة واحدة.
- **الخيط** (Thread / Thread): سلسلة من التعليمات تُنفذ بشكل مستقل. البرنامج قد يملك خيطًا واحدًا أو عدة خيوط.
- **المعالجة المتوازية** (Traitement parallèle / Parallel processing): تنفيذ عدة أجزاء من العمل **في اللحظة نفسها** على أنوية مختلفة.



تشبيه من الحياة اليومية

مطبخ فيه طباخ واحد (نواة واحدة) يحضّر الطبق تلو الآخر. مطبخ فيه 4 طباخين (4 أنوية) يحضّر 4 أطباق في الوقت نفسه. لكنهم يتقاسمون الثلاجة نفسها (L3 وRAM)، وقد ينتظر أحدهم دوره أمامها. وطبق واحد معقد لا يصبح أسرع إلا إذا استطعنا تقسيمه إلى مهام مستقلة.

2.2.5 Hyper-Threading: نواة واحدة لخيطين

تقنية **تعدد الخيوط المتزامن** (Multithreading simultané / SMT, Hyper-Threading) تجعل النواة الواحدة تبدو كأنها نواتان: عندما ينتظر خيط معطيات من الذاكرة، يستعمل الخيط الثاني وحدات الحساب الفارغة. لهذا يُظهر معالجنا 8 «معالجات منطقية» رغم أنه يملك 4 أنوية فقط. لكن إذا كان الخيطان يحتاجان وحدة الحساب نفسها دائمًا، فلا نربح شيئًا تقريبًا، كما سنرى في التجربة.

3.2.5 تجربة: العمل نفسه على 1 و2 و4 و8 خيوط

البرنامج الآتي يحسب مجموع الجذور التربيعية لـ400 مليون عدد، ويقسم العمل بين k خيوط:

```c
/* The same work shared between 1, 2, 4 and 8 threads */
#include <stdio.h>
#include <math.h>
#include <pthread.h>
#include <time.h>

#define WORK 400000000L
static int k;                       /* number of threads */
static double part[8];              /* result of each thread */

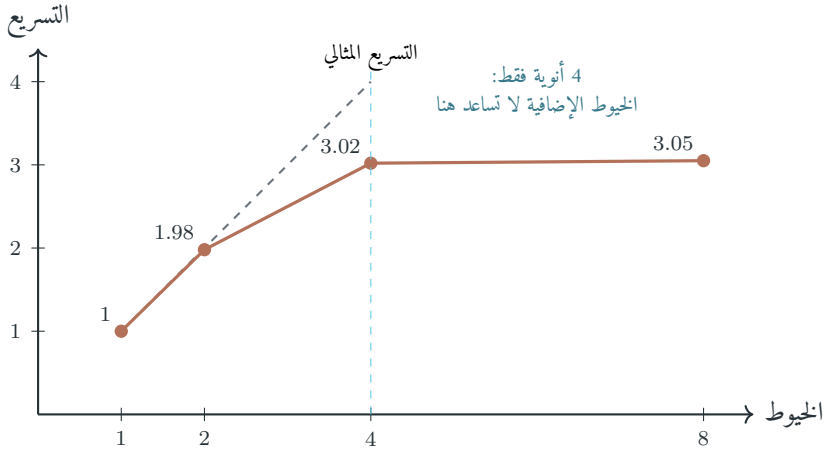
static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

static void *worker(void *arg)      /* thread id computes its part */
{
    long id = (long) arg;
    double s = 0;
    for (long i = id; i < WORK; i += k)
        s += sqrt((double) i);
    part[id] = s;
    return NULL;
}

int main(void)
{
    double t1 = 0;
    for (k = 1; k <= 8; k *= 2) {
        pthread_t th[8];
        double t = now(), total = 0;
        for (long id = 0; id < k; id++)
            pthread_create(&th[id], NULL, worker, (void *) id);
        for (int id = 0; id < k; id++) {
            pthread_join(th[id], NULL);
            total += part[id];
        }
        t = now() - t;
        if (k == 1) t1 = t;
        printf("%d thread(s): %.3f s  speedup %.2f\n", k, t, t1 / t);
    }
    return 0;
}
```

```
screen
1 thread(s): 0.224 s  speedup 1.00
2 thread(s): 0.113 s  speedup 1.98
4 thread(s): 0.074 s  speedup 3.02
8 thread(s): 0.073 s  speedup 3.05
```

السطر	ماذا يفعل؟
18–26	كل خيط يأخذ الأعداد id، $id+k$، $id+2k$...فيحسب جزءًا من المجموع لا يتداخل مع الآخرين.
34–35	`pthread_create`: نطلق k خيوط، يعمل كل واحد منها على نواة.
36–39	`pthread_join`: ننتظر انتهاء كل خيط، ثم نجمع النتائج الجزئية.
40–42	التسريع = الزمن بخيط واحد ÷ الزمن بـk خيوط.



خطوة بخطوة: كيف نقرأ هذه النتائج؟

1) **خيطان**: التسريع 1.98، أي قريب جدًا من 2. العمل انقسم إلى نصفين مستقلين تمامًا.
2) **4 خيوط**: التسريع 3.02 وليس 4. عندما تعمل الأنوية الأربعة معًا يسخن المعالج، فيخفض تردده تلقائيًا. وجزء من البرنامج (إطلاق الخيوط وجمع النتائج) لا يتوازى.
3) **8 خيوط**: التسريع 3.05، تقريبًا مثل 4 خيوط. معالجنا فيه 4 أنوية حقيقية فقط، والخيوط الإضافية تتقاسم وحدات الحساب نفسها، فلا يتقدم العمل أسرع.

4.2.5 قانون Amdahl: حدود التوازي

معظم البرامج فيها جزء **لا يمكن توزيعه** (قراءة الملف، عرض النتيجة، انتظار المستعمل...).

تعريف

إذا كان الجزء القابل للتوازي من البرنامج هو p (بين 0 و1)، ولدينا n نواة، فإن التسريع الأقصى هو:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}}$$

خطوة بخطوة: برنامج 90% منه قابل للتوازي

$p = 0.9$، والجزء التسلسلي $1 - p = 0.1$.

1) بـ4 أنوية: $S = \frac{1}{0.1 + \frac{0.9}{4}} = \frac{1}{0.1 + 0.225} = \frac{1}{0.325} \approx 3.08$.

2) بـ16 نواة: $S = \frac{1}{0.1 + 0.05625} \approx 6.4$.

3) بعدد لا نهائي من الأنوية: $\frac{p}{n} \to 0$، فيصبح $S = \frac{1}{0.1} = 10$.

مهما أضفنا أنوية، لن يكون البرنامج أسرع من 10 مرات، لأن الـ10% التسلسلية تبقى كما هي.

هل تعلم؟

لاحظ أن قانون Amdahl مع $p = 0.9$ و4 أنوية يعطي 3.08، وهو قريب جدًا من التسريع 3.02 الذي قسناه! القانون البسيط يشرح الواقع جيدًا.

5.2.5 الفوائد والتحديات

الفوائد	التحديات
أداء أكبر دون رفع التردد	يجب أن يُكتب البرنامج بخيوط متعددة، وهذا أصعب
استهلاك طاقة أقل لنفس العمل	الجزء التسلسلي يحد من التسريع (قانون Amdahl)
تشغيل عدة برامج في الوقت نفسه بسلاسة	**التزامن**: خيطان يكتبان في المتغير نفسه قد يفسدان النتيجة
إذا تعطلت مهمة، تبقى الأنوية الأخرى تعمل	**تماسك الذاكرة المخبئية**: يجب أن ترى كل الأنوية القيمة نفسها للمتغير

مثال: مشكلة التزامن

خيطان ينفذان `x = x + 1` على المتغير نفسه، و$x = 5$ في البداية:

1) الخيط 1 يقرأ $x = 5$.

2) الخيط 2 يقرأ $x = 5$ أيضًا (قبل أن يكتب الأول).

3) الخيط 1 يحسب $5 + 1$ ويكتب 6.

4) الخيط 2 يحسب $5 + 1$ ويكتب 6 أيضًا.

النتيجة 6 بدل 7: ضاعت زيادة واحدة! لهذا نستعمل **الأقفال** (mutex) أو تعليمات **ذرّية** (`lock add` في x86) تمنع خيطين من تعديل المتغير في الوقت نفسه. في برنامجنا تجنبنا المشكلة: كل خيط يكتب في خانته الخاصة `part[id]`.

3.5 الذاكرة المخبئية وتسلسل الذاكرة

1.3.5 المشكلة: الذاكرة بطيئة جدًا

المعالج قادر على تنفيذ عدة تعليمات في كل نانوثانية، لكن قراءة الذاكرة الرئيسية تحتاج حوالي 100 نانوثانية. لو انتظر المعالج الذاكرة في كل مرة، لبقي معظم الوقت دون عمل.

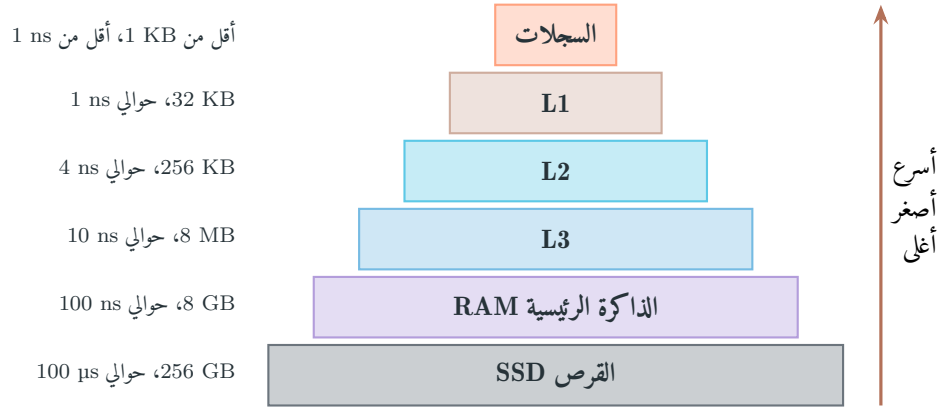
تعريف

الذاكرة المخبئية (Mémoire cache, antémémoire / Cache memory): ذاكرة صغيرة وسريعة جدًا (من نوع SRAM) قريبة من المعالج، تحفظ نسخة من المعطيات التي استُعملت مؤخرًا أو التي سنحتاجها قريبًا.

تشبيه من الحياة اليومية

عندما تراجع للامتحان، لا تضع كل كتب المكتبة على مكتبك. تضع على المكتب الكتب التي تحتاجها الآن (L1)، وفي الحقيبة كتب الأسبوع (L2)، وفي خزانة البيت كتب السنة (L3)، والباقي في المكتبة البعيدة (RAM). كلما كان المكان أقرب، كان أصغر وأسرع.

2.3.5 تسلسل الذاكرة



3.3.5 تجربة: نرى طبقات الذاكرة بأعيننا

البرنامج الآتي يقرأ جدولًا بترتيب عشوائي (كل قراءة تعطي عنوان القراءة التالية، فلا يستطيع المعالج التخمين)، ويقيس متوسط زمن القراءة الواحدة، لأحجام مختلفة من الجدول:

```c
/* Average time of one memory access, for different data sizes.
   Each access depends on the previous one (pointer chasing),
   so the processor cannot guess the next address. */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

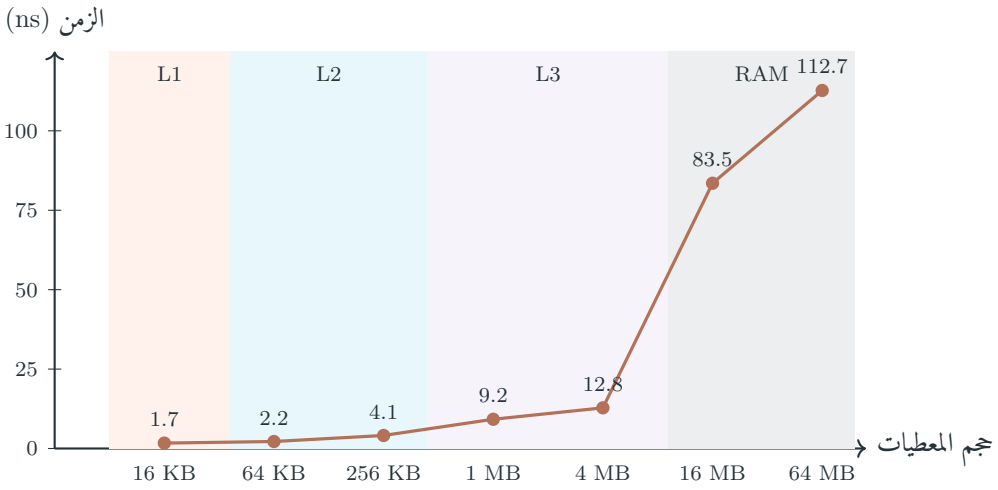
static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

int main(void)
{
    for (long kb = 16; kb <= 65536; kb *= 4) {
        long n = kb * 1024 / sizeof(long);
        long *next = malloc(n * sizeof(long));
        long *order = malloc(n * sizeof(long));
        for (long i = 0; i < n; i++) order[i] = i;
        for (long i = n - 1; i > 0; i--) {          /* random order */
            long j = rand() % (i + 1), t = order[i];
            order[i] = order[j]; order[j] = t;
        }
        for (long i = 0; i < n; i++)                /* one big cycle */
            next[order[i]] = order[(i + 1) % n];

        long p = 0, steps = 20000000;
        double t0 = now();
        for (long s = 0; s < steps; s++)
            p = next[p];
        double ns = (now() - t0) / steps * 1e9;
        printf("%6ld KB : %5.1f ns%s\n", kb, ns, p < 0 ? "!" : "");
        free(next); free(order);
    }
    return 0;
}
```

screen

```
    16 KB :   1.7 ns
    64 KB :   2.2 ns
   256 KB :   4.1 ns
  1024 KB :   9.2 ns
  4096 KB :  12.8 ns
 16384 KB :  83.5 ns
 65536 KB : 112.7 ns
```



خطوة بخطوة: ماذا نرى في المنحنى؟

1) **حتى 32 KB**: الجدول كله في L1، والقراءة تستغرق 1.7 ns فقط.
2) **64 KB و256 KB**: الجدول لم يعد يدخل في L1، فنقرأ من L2: الزمن يرتفع إلى 2 ثم 4 ns.
3) **1 MB و4 MB**: نقرأ من L3 المشتركة: حوالي 9 إلى 13 ns.
4) **16 MB فأكثر**: الجدول أكبر من L3 (8 MB)، فنذهب إلى RAM: الزمن يقفز إلى 84 ثم 113 ns، أي أبطأ بحوالي **65 مرة** من L1!

الدرجات في المنحنى تقع بالضبط عند أحجام الذاكرات المخبئية لمعالجنا.

4.3.5 مبدأ المحلية: لماذا تنجح الذاكرة المخبئية؟

الذاكرة المخبئية صغيرة، لكنها تحتوي في الغالب على ما نحتاجه، بفضل سلوكين طبيعيين في البرامج:

- **المحلية الزمنية** (Localité temporelle / Temporal locality): المعطاة التي استعملناها الآن سنستعملها غالبًا **مرة أخرى قريبًا** (عدّاد الحلقة، المجموع...).
- **المحلية المكانية** (Localité spatiale / Spatial locality): بعد قراءة عنوان، نقرأ غالبًا **العناوين المجاورة** (عناصر جدول، تعليمات متتالية).

تعريف

سطر الذاكرة المخبئية (Ligne de cache / Cache line): الذاكرة المخبئية لا تنقل بايتًا واحدًا، بل كتلة كاملة من **64 بايتًا** مجاورة. عندما نقرأ `t[0]`، تصل معه `t[1]` إلى `t[15]` (للأعداد من 4 بايتات) مجانًا.

5.3.5 تجربة: ترتيب القراءة يغيّر كل شيء

```c
/* Same sum, two orders: row by row, or column by column */
#include <stdio.h>
#include <time.h>

#define N 4096
static int m[N][N];                 /* 4096 x 4096 ints = 64 MB */

static double now(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec * 1e-9;
}

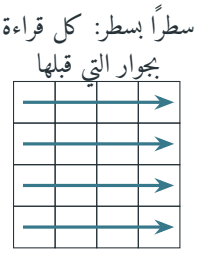
int main(void)
{
    long s = 0;
    double t;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            m[i][j] = 1;

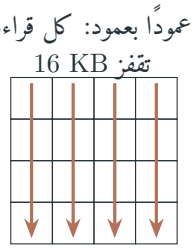
    t = now();
    for (int i = 0; i < N; i++)      /* row by row: neighbours */
        for (int j = 0; j < N; j++)
            s += m[i][j];
    printf("rows    : %.3f s\n", now() - t);

    t = now();
    for (int j = 0; j < N; j++)      /* column by column: jumps */
        for (int i = 0; i < N; i++)
            s += m[i][j];
    printf("columns : %.3f s\n", now() - t);
    printf("sum = %ld\n", s);
    return 0;
}
```

screen

```
rows    : 0.009 s
columns : 0.301 s
sum = 33554432
```





خطوة بخطوة: لماذا الأعمدة أبطأ بـ33 مرة؟

1) في لغة C، الجدول `m[4096][4096]` محفوظ **سطرًا بعد سطر**: `m[0][0]` ثم `m[0][1]` ثم...

2) **سطرًا بسطر**: كل سطر ذاكرة مخبئية (64 بايتًا) يحتوي على 16 عددًا متجاورًا. نقرأ من RAM مرة، ثم نستعمل 16 عددًا من الذاكرة المخبئية. والمعالج يلاحظ أننا نتقدم بانتظام فيجلب الأسطر القادمة مسبقًا.

3) **عمودًا بعمود**: `m[0][j]` و`m[1][j]` بينهما $4096 \times 4 = 16\,384$ بايتًا. كل قراءة في سطر ذاكرة مخبئية جديد، فنستعمل عددًا واحدًا فقط من كل 16 عددًا نقلناها.

4) النتيجة الحقيقية: 0.009 s مقابل 0.301 s، أي حوالي **33 مرة** أبطأ، لنفس العمليات تمامًا!

6.3.5 الإصابة والإخفاق، وزمن الوصول المتوسط

تعريف

- **الإصابة** (Succès / Hit): المعطاة موجودة في الذاكرة المخبئية.
- **الإخفاق** (Défaut / Miss): المعطاة غير موجودة، فنذهب إلى المستوى التالي (الأبطأ).
- **معدل الإصابة** h: نسبة القراءات التي وجدت معطياتها في الذاكرة المخبئية.

$$\text{الزمن المتوسط} = h \times t_{cache} + (1 - h) \times t_{RAM}$$

خطوة بخطوة: $t_{cache} = 1$ ns، $t_{RAM} = 100$ ns

1) معدل إصابة 90%: $0.9 \times 1 + 0.1 \times 100 = 0.9 + 10 = 10.9$ ns.

2) معدل إصابة 99%: $0.99 \times 1 + 0.01 \times 100 = 0.99 + 1 = 1.99$ ns.

الانتقال من 90% إلى 99% يجعل الذاكرة **أسرع بأكثر من 5 مرات**. لهذا تهتم المعالجات الحديثة جدًا بتحسين معدل الإصابة.

7.3.5 أين نضع المعطاة في الذاكرة المخبئية؟ (التطابق)

الذاكرة المخبئية مقسمة إلى أسطر. علينا أن نقرر في أي سطر نضع كتلة قادمة من الذاكرة:

النوع	القاعدة	الميزة والعيب
التطابق المباشر (direct mapped)	كل كتلة لها سطر واحد ممكن: رقم الكتلة mod عدد الأسطر	بسيط وسريع، لكن كتلتان قد تتصارعان على السطر نفسه
التجميعي الكامل (fully associative)	أي كتلة في أي سطر	أقل صراعات، لكن البحث في كل الأسطر مكلف
التجميعي بمجموعات (set associative)	الكتلة تذهب إلى مجموعة محددة، وفيها إلى أي سطر من k أسطر	الحل الوسط المستعمل في كل المعالجات الحديثة

خطوة بخطوة: تقسيم عنوان في ذاكرة مخبئية بتطابق مباشر

ذاكرة مخبئية سعتها 1 KB، وأسطرها من 16 بايتًا، والعناوين من 16 بتًا.

1) عدد الأسطر: $1024 \div 16 = 64 = 2^6$ سطرًا، إذن نحتاج **6 بتات** لرقم السطر.

2) داخل السطر 16 بايتًا $= 2^4$، إذن **4 بتات** للإزاحة.

3) الباقي: $16 - 6 - 4 = 6$ بتات، وتسمى **الوسم** (tag): تحدد أي كتلة من الذاكرة موجودة الآن في السطر.



مثال: $1A2C_H = 000110\,100010\,1100_2$. الوسم $000110_2 = 6$، والسطر $100010_2 = 34$، والإزاحة $1100_2 = 12$.

انتبه

العنوان الثنائي يُكتب دائمًا من اليسار إلى اليمين، حتى في كتاب بالعربية: البتات العليا (الوسم) على اليسار والبت 0 على اليمين، تمامًا كما رسمناه في الشكل.

8.3.5 الإستبدال والكتابة

- **سياسة الاستبدال**: عندما تمتلئ المجموعة، أي سطر نطرد؟ غالبًا LRU (Least Recently Used): السطر الذي لم نستعمله منذ أطول مدة.
- **سياسة الكتابة**:
  - Write-through: نكتب في الذاكرة المخبئية وفي RAM معًا. بسيط لكنه بطيء.
  - Write-back: نكتب في الذاكرة المخبئية فقط، ولا نكتب في RAM إلا عندما نطرد السطر. أسرع، وهو المستعمل اليوم.

هل تعلم؟

تذكر نموذج هارفارد من الفصل الأول: ذاكرة L1 مقسمة إلى جزأين، L1i للتعليمات وL1d للمعطيات (في معالجنا 32 KB لكل واحدة)، فيستطيع المعالج جلب تعليمة وقراءة معطاة في الوقت نفسه. أما L2 وL3 وRAM فمشتركة بينهما، كما في نموذج فون نيومان.

4.5 تمارين

تمرين 1.5

برنامج 75% منه قابل للتوازي. احسب التسريع بـ2 و4 و8 أنوية، والتسريع الأقصى.

الحل

$1 - p = 0.25$. $S(2) = \dfrac{1}{0.25 + 0.375} = 1.6$، $S(4) = \dfrac{1}{0.25 + 0.1875} \approx 2.29$، $S(8) = \dfrac{1}{0.25 + 0.09375} \approx 2.91$. الأقصى $= \dfrac{1}{0.25} = 4$.

تمرين 2.5

ذاكرة مخبئية زمنها 2 ns، والذاكرة الرئيسية 80 ns، ومعدل الإصابة 95%. ما الزمن المتوسط؟

الحل

$0.95 \times 2 + 0.05 \times 80 = 1.9 + 4 = 5.9$ ns.

تمرين 3.5

ذاكرة مخبئية بتطابق مباشر: 4 KB، أسطر من 32 بايتًا، عناوين من 32 بتًا. كم بتًا للإزاحة وللسطر وللوسم؟

الحل

الإزاحة: $2^5 = 32$، إذن 5 بتات. الأسطر: $4096 \div 32 = 128 = 2^7$، إذن 7 بتات. الوسم: $32 - 7 - 5 = 20$ بتًا.

تمرين 4.5

اشرح لماذا يكون هذا الكود بطيئًا، وكيف نصلحه:

```
for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        s += m[i][j];
```

الحل

يقرأ الجدول عمودًا بعمود، فكل قراءة في سطر ذاكرة مخبئية مختلف (لا محلية مكانية). نصلحه بتبديل الحلقتين: `for (j ...) for (i ...)`، فنقرأ سطرًا بسطر.

تمرين 5.5

لماذا لم يتحسن البرنامج `threads.c` عند الانتقال من 4 إلى 8 خيوط في معالجنا؟

الحل

لأن المعالج يملك 4 أنوية حقيقية فقط. الخيوط 5 إلى 8 تعمل بتقنية Hyper-Threading على الأنوية نفسها، والبرنامج يستعمل وحدة الحساب طوال الوقت، فلا توجد وحدات فارغة يستفيد منها الخيط الثاني.

الملحق ا

ملخص تعليمات 8086

هذا الملحق يجمع في مكان واحد كل التعليمات التي درسناها في الفصل الثالث، لتعود إليها بسرعة أثناء كتابة برامجك. الرموز: d الوجهة، s المصدر، r سجل، m خانة ذاكرة.

وظائف DOS الأكثر استعمالًا (int 21h)

AH	الوظيفة	المدخلات / المخرجات
$_H$01	قراءة حرف مع إظهاره	المخرج في AL
$_H$02	كتابة حرف	الحرف في DL
$_H$09	كتابة سلسلة تنتهي بـ$	العنوان في DX
$_H$0A	قراءة سطر في ذاكرة مؤقتة	العنوان في DX
$_H$4C	إنهاء البرنامج	الرمز في AL

التعليمات

التعليمة	المعنى	الأعلام المتأثرة
النقل		
`mov d, s`	$d \leftarrow s$	لا شيء
`xchg a, b`	تبادل a وb	لا شيء
`lea r, m`	r يأخذ عنوان m	لا شيء
`push s`	$SP \leftarrow SP - 2$، ثم $[SP] \leftarrow s$	لا شيء
`pop d`	$d \leftarrow [SP]$، ثم $SP \leftarrow SP + 2$	لا شيء
`out port, al` / `in al, port`	قراءة من منفذ / كتابة في منفذ	لا شيء
الحساب		
`add d, s`	$d \leftarrow d + s$	كلها
`adc d, s`	$d \leftarrow d + s + CF$	كلها
`sub d, s`	$d \leftarrow d - s$	كلها
`sbb d, s`	$d \leftarrow d - s - CF$	كلها
`inc d`	$d \leftarrow d + 1$	كلها إلا CF
`dec d`	$d \leftarrow d - 1$	كلها إلا CF
`neg d`	$d \leftarrow -d$	كلها
`cmp a, b`	يحسب a - b دون حفظ	كلها
`mul s` (8 بتات)	$AX \leftarrow AL * s$	OF ،CF
`mul s` (16 بتًا)	$DX{:}AX \leftarrow AX * s$	OF ،CF
`div s` (8 بتات)	$AL \leftarrow AX \,/\, s$، $AH \leftarrow AX \bmod s$	غير محددة
`div s` (16 بتًا)	$AX \leftarrow DX{:}AX \,/\, s$، $DX \leftarrow DX{:}AX \bmod s$	غير محددة
المنطق والإزاحة		
`and` ،`or` ،`xor`	عمليات بتًا ببت	SF ،ZF ،$CF = OF = 0$
`not d`	قلب كل البتات	لا شيء
`test a, b`	`and` دون حفظ	مثل `and`
`shr d, n` / `shl d, n`	إزاحة: ضرب أو قسمة على 2^n	CF = آخر بت خرج
`ror d, n` / `rol d, n`	تدوير	CF
التحكم في التدفق		
`jmp L`	قفز دائمًا	لا شيء
`jnz` ،`jz` ،`jne` ،`je`	حسب ZF	--
`jle` ،`jl` ،`jge` ،`jg`	مقارنة **بإشارة**	--
`jbe` ،`jb` ،`jae` ،`ja`	مقارنة **دون إشارة**	--
`loop L`	$CX \leftarrow CX - 1$، ثم القفز إذا كان $CX \neq 0$	لا شيء
`jcxz L`	القفز إذا كان $CX = 0$	--
`ret` / `call P`	استدعاء إجراء / العودة منه	لا شيء
`iret` / `int n`	مقاطعة / العودة منها	$IF = TF = 0$ / تُستعاد
السلاسل		
`movsw` / `movsb`	$DS{:}[SI] \rightarrow ES{:}[DI]$، ثم تقدم SI وDI	لا شيء
`rep`	كرر التعليمة CX مرة	--
`std` / `cld`	$DF = 0$ (تصاعدي) / $DF = 1$	DF

الملحق ب

EMU8086 وNASM: الفروق في الكتابة

برامج الفصل الثالث مكتوبة لـEMU8086 (صيغة قريبة من MASM)، وبرامج الفصل الرابع لـNASM. المعالج هو نفسه، لكن المجمّعين لا يكتبان بعض الأشياء بالطريقة نفسها:

ما نريد	EMU8086 / MASM	NASM
عنوان متغير في سجل	`mov dx, offset msg` أو `lea dx, msg`	`mov dx, msg`
قيمة متغير	`mov al, [x]` أو `mov al, x`	`mov al, [x]` (الأقواس إجبارية)
تحديد الحجم	`mov byte ptr [bx], 5`	`mov byte [bx], 5`
عدد ست عشري	`0FFh`	`0FFh` أو `0xFF`
إجراء	`p proc` ... `p endp`	تسمية بسيطة: `p:`
ثابت	`N equ 10`	`N equ 10`
جدول من 10 أصفار	`t db 10 dup(0)`	`t times 10 db 0`
التجميع	زر Emulate	`nasm -f bin prog.asm -o prog.com`

> **انتبه**
>
> عادة جيدة: اكتب دائمًا الأقواس `[ ]` عندما تقصد محتوى الذاكرة، حتى في EMU8086. هكذا يبقى برنامجك واضحًا، ويعمل مع المجمّعين بتعديلات قليلة. هذا ما فعلناه في كل برامج هذا الدرس.

الملحق ج

قاموس المصطلحات

English	Français	بالعربية
Computer	Ordinateur	الحاسوب
Processor, CPU	Processeur, UCT	المعالج
Central unit	Unité centrale	الوحدة المركزية
Main memory	Mémoire centrale	الذاكرة المركزية
Volatile memory	Mémoire volatile	ذاكرة متطايرة
Bus	Bus	الحافلة
Address bus	Bus d'adresses	حافلة العناوين
Data bus	Bus de données	حافلة المعطيات
Control bus	Bus de contrôle	حافلة التحكم
Register	Registre	السجل
Program counter (PC)	Compteur ordinal (CO)	العدّاد البرنامجي
Instruction register (IR)	Registre d'instruction (RI)	سجل التعليمة
Memory address register (MAR)	Registre d'adresse mémoire (RAM)	سجل عنوان الذاكرة
Memory data register (MDR)	Registre d'information mémoire (RIM)	سجل معطيات الذاكرة
Accumulator	Accumulateur	المركّم
Flag	Indicateur, drapeau	العلم
Arithmetic logic unit (ALU)	Unité arithmétique et logique (UAL)	وحدة الحساب والمنطق
Control unit	Unité de commande (UC)	وحدة التحكم
Decoder	Décodeur	المفكك
Sequencer	Séquenceur	المتتالي
Half adder	Demi-additionneur	نصف المجمّع
Full adder	Additionneur complet	المجمّع الكامل
Two's complement	Complément à deux	المتمم لاثنين
Instruction set	Jeu d'instructions	مجموعة التعليمات
Opcode	Code opération	الرمز العملي
Addressing mode	Mode d'adressage	نمط العنونة
Effective address	Adresse effective	العنوان الفعلي
Physical address	Adresse physique	العنوان الفيزيائي
Segment, offset	Segment, déplacement	المقطع والإزاحة
Instruction cycle	Cycle d'instruction	دورة التعليمة

English	Français	بالعربية
Fetch, decode, execute	Recherche, décodage, exécution	الجلب، فك الترميز، التنفيذ
Assembly language	Langage d’assemblage	لغة التجميع
Assembler	Assembleur	المجمّع
Stack	Pile	المكدس
Procedure, subroutine	Procédure, sous-programme	الإجراء
Interrupt	Interruption	المقاطعة
Interrupt vector table	Table des vecteurs d’interruption	جدول أشعة المقاطعات
I/O port	Port d’E/S	المنفذ
Protected mode	Mode protégé	النمط المحمي
Flat memory model	Modèle mémoire plat	النموذج المسطح
Pipeline	Pipeline	خط الأنابيب
Branch prediction	Prédiction de branchement	التنبؤ بالقفز
Core	Cœur	النواة
Thread	Thread, fil d’exécution	الخيط
Cache memory	Mémoire cache, antémémoire	الذاكرة المخبئية
Cache line	Ligne de cache	سطر الذاكرة المخبئية
Hit / Miss	Succès / Défaut	الإصابة / الإخفاق
Temporal and spatial locality	Localité temporelle et spatiale	المحلية الزمنية والمكانية

المراجع

(1 A. S. Tanenbaum, *Structured Computer Organization*, Pearson, 2012.

(2 D. A. Patterson, J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.

(3 P. Zanella, Y. Ligier, E. Lazard, *Architecture et technologie des ordinateurs*, 6e édition, Dunod, 2018.

(4 B. B. Brey, *The Intel Microprocessors: 8086/8088 … Core2 with 64-bit Extensions*, Pearson.

(5 EMU8086, Microprocessor Emulator.

(6 NASM, The Netwide Assembler: https://www.nasm.us/

(7 MASM, Microsoft Macro Assembler reference: https://learn.microsoft.com/en-us/cpp/assembler/masm/

(8 GAS, The GNU Assembler: https://sourceware.org/binutils/docs/as/